

salmon-disease-detection

November 30, 2025

1 Salmon Disease Detection using CNN

1.1 Course Project - Aquaculture05

Project Overview:

- Build 3 CNN models with varying architectures (1-3 convolutional layers) + 1 Ensemble
- Use Stratified 80/20 Train-Test Split with Multiple Seeds for robust evaluation
- Apply Six Sigma DMAIC methodology to analyze model quality
- Dataset: 50 images of Fresh fish, 91 images of Infected fish (before augmentation)

Author: Gabriel Temitayo Fayomi, Mads Teien

Date: November 2025

1. Import Libraries and Setup

```
[ ]: # =====  
# IMPORTS AND SETUP  
# =====  
# Essential libraries for data manipulation and visualization  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from pathlib import Path  
import os  
import random  
import gc  
import warnings  
warnings.filterwarnings('ignore')  
  
# Deep Learning libraries  
import tensorflow as tf  
from tensorflow import keras  
from tensorflow.keras import layers, models  
from tensorflow.keras import backend as K  
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img,   
↳img_to_array
```

```

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, \
    ModelCheckpoint

# Scikit-learn libraries
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)

print("="*80)
print("SALMON DISEASE DETECTION PROJECT")
print("Deep Learning + Six Sigma DMAIC Analysis")
print("="*80)

# =====
# HYPERPARAMETERS
# =====
# Seed for reproducibility - this is the "default" seed for general operations
SEED = 42

# Seeds for multi-seed analysis (for statistical validity)
EXPERIMENT_SEEDS = [i for i in range(30)] # 30 different seeds

# Image processing settings
IMG_WIDTH = 400 # Match original image dimensions
IMG_HEIGHT = 166
IMG_CHANNELS = 3

# Training settings
BATCH_SIZE = 32
EPOCHS = 50
LEARNING_RATE = 0.001
TEST_SIZE = 0.2

# Augmentation settings
AUGMENTATIONS_PER_IMAGE = 4

# Set random seeds for initial reproducibility
np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

# =====
# FIGURE STYLING - Report-quality settings
# =====
# Font sizes scaled UP for multi-panel figures that get shrunk in LaTeX reports
BASE_FONT = {

```

```

    'title': 24,          # Increased from 12
    'suptitle': 28,       # Increased from 14
    'label': 22,          # Increased from 10
    'tick': 20,           # Increased from 8
    'legend': 18,         # Increased from 9
    'annotation': 18,     # Increased from 8
}

def get_font_scale(n_rows, n_cols):
    """Scale fonts based on number of panels - more panels = bigger fonts,
    ↪needed."""
    n_panels = n_rows * n_cols
    if n_panels == 1:
        return 1.0
    elif n_panels == 2:
        return 1.3
    elif n_panels <= 4:
        return 1.5
    elif n_panels <= 6:
        return 1.7
    elif n_panels <= 9:
        return 1.8
    else:
        return 2.0

def get_fonts(n_rows=1, n_cols=1):
    """Get scaled font sizes for a figure with given layout."""
    scale = get_font_scale(n_rows, n_cols)
    return {k: int(v * scale) for k, v in BASE_FONT.items()}

# Set matplotlib defaults for clean, professional figures
# Global matplotlib settings for publication/report-quality figures
# Font sizes scaled UP for multi-panel figures that get shrunk in LaTeX reports
plt.rcParams.update({
    'figure.facecolor': 'white',
    'axes.facecolor': 'white',
    'axes.grid': True,
    'grid.alpha': 0.3,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 100,
    'savefig.dpi': 300,
    'savefig.bbox': 'tight',
    'savefig.facecolor': 'white',
    'font.family': 'sans-serif',
})

```

```

# Ensure figures directory exists
Path('report/figures').mkdir(parents=True, exist_ok=True)

print(" Libraries imported successfully")
print(" Report-ready figure styling configured")

# =====
# GPU CONFIGURATION - MIXED PRECISION FOR FASTER TRAINING
# =====
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f" GPU available: {gpus[0].name}")

        # Enable mixed precision for faster training on modern GPUs
        from tensorflow.keras import mixed_precision
        mixed_precision.set_global_policy('mixed_float16')
        print(" Mixed precision (FP16) enabled for faster training")

    except RuntimeError as e:
        print(f"GPU configuration error: {e}")
else:
    print("No GPU found, using CPU")

# =====
# HYPERPARAMETERS
# =====
IMG_WIDTH = 128
IMG_HEIGHT = 128
IMG_CHANNELS = 3
BATCH_SIZE = 32
EPOCHS = 50
LEARNING_RATE = 0.001
TEST_SIZE = 0.2

print(f"\n Hyperparameters set:")
print(f" Image size: {IMG_WIDTH}x{IMG_HEIGHT}x{IMG_CHANNELS}")
print(f" Batch size: {BATCH_SIZE}")
print(f" Epochs: {EPOCHS}")
print(f" Learning rate: {LEARNING_RATE}")
print(f" Test split: {TEST_SIZE*100}%")
print(f" Number of seeds for multi-seed analysis: {len(EXPERIMENT_SEEDS)}")

```

```

Libraries imported successfully
Report-ready figure styling configured
GPU available: /physical_device:GPU:0

```

```
INFO:tensorflow:Mixed precision compatibility check (mixed_float16): OK
Your GPU will likely run quickly with dtype policy mixed_float16 as it has
compute capability of at least 7.0. Your GPU: NVIDIA GeForce RTX 3070 Laptop
GPU, compute capability 8.6
Mixed precision (FP16) enabled for faster training
```

```
Hyperparameters set:
Image size: 128x128x3
Batch size: 32
Epochs: 50
Learning rate: 0.001
Test split: 20.0%
Number of seeds for multi-seed analysis: 30
Mixed precision (FP16) enabled for faster training
```

```
Hyperparameters set:
Image size: 128x128x3
Batch size: 32
Epochs: 50
Learning rate: 0.001
Test split: 20.0%
Number of seeds for multi-seed analysis: 30
```

2. Data Loading and Exploration

DMAIC Phase 1: DEFINE

- **Objective:** Develop CNN models to classify salmon as Fresh or Infected
- **Dataset:** 50 images of Fresh fish, 91 images of Infected fish (before augmentation)
- **Target:** Achieve high accuracy with robust model performance
- **Scope:** Compare 3 CNN architectures and an ensemble model, then select the best for deployment

```
[2]: # Define paths
DATA_DIR = Path('Data-SalmonScan')
FRESH_DIR = DATA_DIR / 'FreshFish'
INFECTED_DIR = DATA_DIR / 'InfectedFish'

# Image parameters (as per dataset description)
IMG_WIDTH = 400
IMG_HEIGHT = 166
IMG_CHANNELS = 3

# Training parameters
BATCH_SIZE = 16
EPOCHS = 50
LEARNING_RATE = 0.001
```

```
PREPROCESSING_MODE = 'none'
```

```
[3]: # Load dataset with disease-focused preprocessing
def load_dataset(fresh_dir, infected_dir, preprocessing='none'):
    """
    Load all images and labels from directories with disease-focused_
    preprocessing

    Args:
        fresh_dir: Path to fresh fish images
        infected_dir: Path to infected fish images
        preprocessing: Kept for backward compatibility; currently only
            'none' is supported and images are loaded without any
            disease-specific preprocessing.

    Returns:
        X: Image array (float32 in 0-255 range)
        y: Labels (0=Fresh, 1=Infected)
        filenames: List of filenames
    """
    images = []
    labels = []
    filenames = []

    if preprocessing != 'none':
        preprocessing = 'none'

    # Load fresh fish images (label = 0)
    fresh_files = sorted(list(fresh_dir.glob('*.png')))
    for img_path in fresh_files:
        try:
            img = load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            img_array = img_to_array(img) # Returns 0-255 range

            images.append(img_array)
            labels.append(0) # Fresh = 0
            filenames.append(img_path.name)
        except Exception as e:
            print(f"Error loading {img_path}: {e}")

    # Load infected fish images (label = 1)
    infected_files = sorted(list(infected_dir.glob('*.png')))
    for img_path in infected_files:
        try:
            img = load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            img_array = img_to_array(img) # Returns 0-255 range
```

```

        images.append(img_array)
        labels.append(1) # Infected = 1
        filenames.append(img_path.name)
    except Exception as e:
        print(f"Error loading {img_path}: {e}")

    # Convert to numpy arrays (float32 in 0-255 range)
    X = np.array(images, dtype=np.float32)
    y = np.array(labels)

    return X, y, filenames

# Load the data WITHOUT preprocessing initially (we'll apply it after
↳ augmentation)
# This prevents augmentation from destroying the disease-focused features
X, y, filenames = load_dataset(FRESH_DIR, INFECTED_DIR, preprocessing='none')

```

```

[ ]: # Visualize sample images - simplified 2x3 grid for report
fig, axes = plt.subplots(2, 3, figsize=(10, 4))

# Show 3 fresh fish
fresh_indices = np.where(y == 0)[0]
for i, ax in enumerate(axes[0]):
    idx = fresh_indices[i]
    ax.imshow(np.clip(X[idx] / 255.0, 0, 1))
    ax.axis('off')
axes[0, 0].set_ylabel('Fresh', fontsize=14, fontweight='bold')

# Show 3 infected fish
infected_indices = np.where(y == 1)[0]
for i, ax in enumerate(axes[1]):
    idx = infected_indices[i]
    ax.imshow(np.clip(X[idx] / 255.0, 0, 1))
    ax.axis('off')
axes[1, 0].set_ylabel('Infected', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('report/figures/sample_images.png', dpi=300, bbox_inches='tight',
↳ facecolor='white')
plt.show()

# Class distribution - clean single plot
fig, ax = plt.subplots(figsize=(6, 5))
class_counts = pd.Series(y).value_counts().sort_index()
bars = ax.bar(['Fresh', 'Infected'], class_counts.values, color=['#2ecc71',
↳ '#e74c3c'], alpha=0.8)

```

```

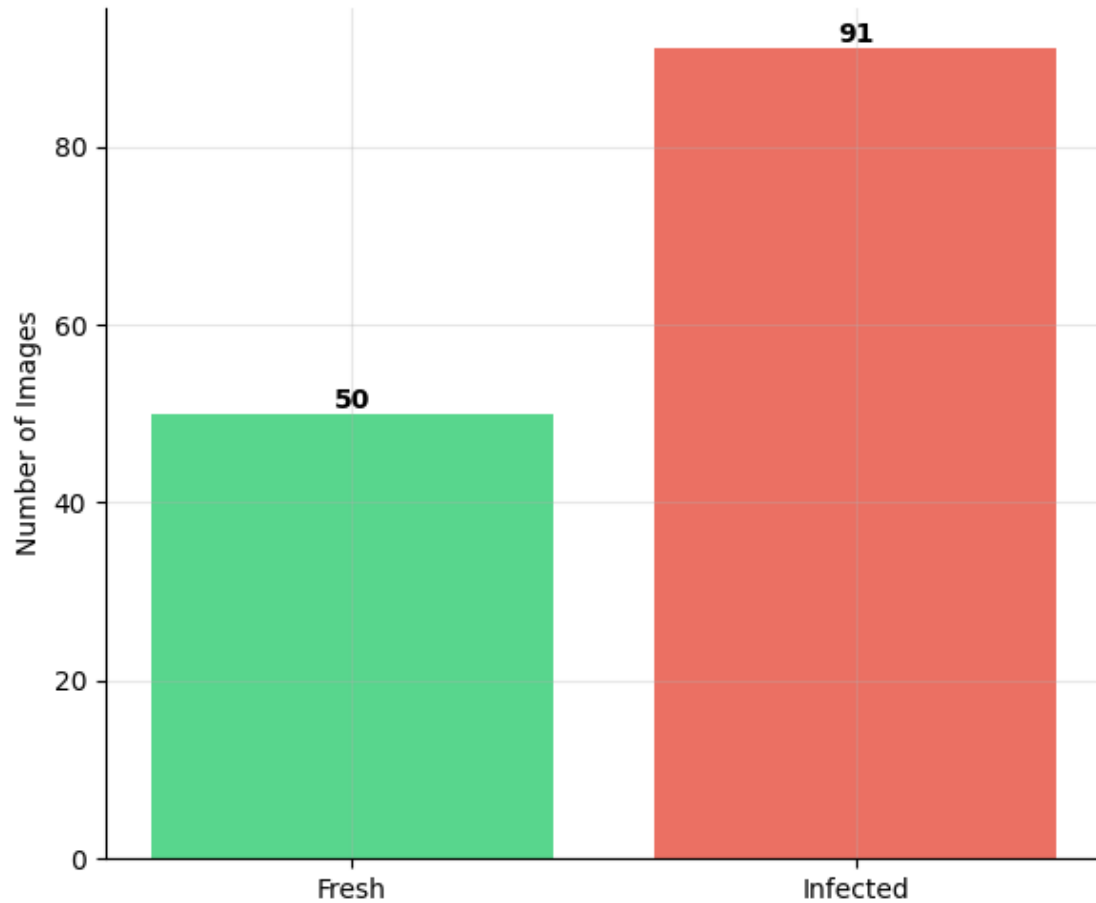
ax.set_ylabel('Number of Images')
ax.grid(axis='y', alpha=0.3)

# Add count labels on bars
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width() / 2.0, height, f'{int(height)}',
            ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.savefig('report/figures/class_distribution.png', dpi=300,
           bbox_inches='tight', facecolor='white')
plt.show()

```





1.1.1 DMAIC Phase 2: MEASURE

Step 1-3: Data Collection and Baseline Metrics

- Split data using Stratified 80/20 Train-Test Split
- Establish baseline performance metrics
- Document data characteristics

```
[5]: # Configure Train-Test Split
TEST_SIZE = 0.2 # 80% train, 20% test

print(f"Configured Stratified Train-Test Split with {int((1-TEST_SIZE)*100)}%_
↳train, {int(TEST_SIZE*100)}% test.")
print(f"Statistical robustness provided by {len(EXPERIMENT_SEEDS)} random seeds:
↳ {EXPERIMENT_SEEDS}")

AUGMENTATIONS_PER_IMAGE = 10 # Keep 10x as requested
```

Configured Stratified Train-Test Split with 80% train, 20% test.
Statistical robustness provided by 30 random seeds: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]

```
[6]: # GENERATE AUGMENTED DATASET
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Create augmentation generator with light geometric transforms (no brightness)
augmentation_datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    vertical_flip=False,
    fill_mode='nearest'
)

def generate_augmented_images(X_class, y_class, n_augmentations, class_name):
    """Generate augmented images for a specific class"""

    X_augmented = []
    y_augmented = []

    # Fit the generator
    augmentation_datagen.fit(X_class)

    n_generated = 0
    batch_size = 32
    target_total = n_augmentations

    # Generate augmented images
    for X_batch, y_batch in augmentation_datagen.flow(X_class, y_class,
                                                    batch_size=batch_size,
                                                    shuffle=True,
                                                    seed=SEED):
        for i in range(len(X_batch)):
            if n_generated >= target_total:
                break
            X_augmented.append(X_batch[i])
            y_augmented.append(y_batch[i])
            n_generated += 1

    if n_generated >= target_total:
        break
```

```
return np.array(X_augmented, dtype=np.float32), np.array(y_augmented)
```

```
[ ]: # =====
# AUGMENTATION VISUALIZATION - Show Original vs Augmented Samples
# =====
# Visualize the augmentation pipeline to understand what transformations are
    ↳ applied

print("="*80)
print("AUGMENTATION VISUALIZATION")
print("="*80)

# Select one sample to show augmentations
infected_sample_idx = np.where(y == 1)[0][0]

# Simple 1x4 grid: Original + 3 Augmented
fig, axes = plt.subplots(1, 4, figsize=(12, 3.5))

# Plot original
axes[0].imshow(X[infected_sample_idx].astype(np.uint8))
axes[0].set_title('Original')
axes[0].axis('off')

# Generate 3 augmentations
img_batch = np.expand_dims(X[infected_sample_idx], axis=0)
aug_iter = augmentation_datagen.flow(img_batch, batch_size=1, seed=SEED)

for i in range(3):
    aug_img = next(aug_iter)[0]
    axes[i+1].imshow(aug_img.astype(np.uint8))
    axes[i+1].set_title(f'Augmented {i+1}')
    axes[i+1].axis('off')

plt.tight_layout()
plt.savefig('report/figures/augmentation_examples.png', dpi=300,
    ↳ bbox_inches='tight', facecolor='white')
plt.show()

print("\nAugmentation transforms applied:")
print("    • Rotation: ±15°")
print("    • Width/Height shift: ±10%")
print("    • Shear: 10%")
print("    • Zoom: ±10%")
print("    • Horizontal flip: Yes")
print("    • Vertical flip: No (fish have consistent orientation)")
print(f"\nAugmentations per original image: {AUGMENTATIONS_PER_IMAGE}x")
```

```
print(f"This increases training data from {len(X)} to ~{len(X) * (1 +  
↪AUGMENTATIONS_PER_IMAGE)} samples")
```

AUGMENTATION VISUALIZATION



Augmentation transforms applied:

- Rotation: $\pm 15^\circ$
- Width/Height shift: $\pm 10\%$
- Shear: 10%
- Zoom: $\pm 10\%$
- Horizontal flip: Yes
- Vertical flip: No (fish have consistent orientation)

Augmentations per original image: 10x

This increases training data from 141 to ~1551 samples

3. Model Architecture Definition

DMAIC Phase 3: ANALYZE

Step 4-6: Design Three CNN Architectures + Ensemble

We will build and compare 3 CNN models with increasing complexity and then an ensemble:

- **Model 1:** 1 Conv layer + 1 MaxPool + 1 Dense (Fully Connected)
- **Model 2:** 2 Conv layers + 2 MaxPool + 1 Dense
- **Model 3:** 3 Conv layers + 3 MaxPool + 1 Dense

```
[8]: def create_cnn_model(num_conv_layers, input_shape=(IMG_HEIGHT, IMG_WIDTH,  
↪IMG_CHANNELS)):
    """Create a CNN model with configurable number of convolutional blocks."""
    model = models.Sequential(name=f'CNN_Model_{num_conv_layers}_Layers')

    # First convolutional block
    model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same',  
                           input_shape=input_shape, name='conv1'))
    model.add(layers.BatchNormalization(name='bn1'))
    model.add(layers.MaxPooling2D((2, 2), name='maxpool1'))
    model.add(layers.Dropout(0.25, name='dropout1'))
```

```

# Additional convolutional blocks based on num_conv_layers
if num_conv_layers >= 2:
    model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same',
↪name='conv2'))
    model.add(layers.BatchNormalization(name='bn2'))
    model.add(layers.MaxPooling2D((2, 2), name='maxpool2'))
    model.add(layers.Dropout(0.25, name='dropout2'))

if num_conv_layers >= 3:
    model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same',
↪name='conv3'))
    model.add(layers.BatchNormalization(name='bn3'))
    model.add(layers.MaxPooling2D((2, 2), name='maxpool3'))
    model.add(layers.Dropout(0.3, name='dropout3'))

# Flatten and fully connected layers
model.add(layers.Flatten(name='flatten'))
#model.add(layers.Dense(256, activation='relu', name='dense1')) #
↪Increased from 128
#model.add(layers.BatchNormalization(name='bn_dense'))
#model.add(layers.Dropout(0.5, name='dropout_dense'))
model.add(layers.Dense(1, activation='sigmoid', dtype='float32',
↪name='output')) # dtype for mixed precision

# Compile model
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
    loss='binary_crossentropy',
    metrics=['accuracy', keras.metrics.Precision(), keras.metrics.Recall()]
)

return model

```

4. Model Training

DMAIC Phase 3: ANALYZE (Continued)

Step 7-9: Train Models and Collect Performance Data

```

[9]: import gc
from tensorflow.keras import backend as K
from sklearn.utils.class_weight import compute_class_weight
import os

# Define callbacks for training
early_stopping = EarlyStopping(
    monitor='val_loss',

```

```

        patience=10,
        restore_best_weights=True,
        verbose=0
    )

    reduce_lr = ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        min_lr=1e-7,
        verbose=0
    )

    callbacks = [early_stopping, reduce_lr]

# Model configurations
    model_configs = {
        'Model_1': 1,
        'Model_2': 2,
        'Model_3': 3
    }

# Storage for statistical analysis across multiple seeds
    multi_run_results = []

# Ensure models directory exists
    os.makedirs('models', exist_ok=True)

# Storage for all trained models paths (for later analysis)
    saved_model_paths = {} # {(model_name, seed): path}

# Storage for test sets from each seed (to fix evaluation bug)
    test_sets = {} # {seed_idx: (X_test, y_test)}

    print("\n" + "="*80)
    print(f"STARTING MULTI-SEED EXPERIMENT (Seeds: {EXPERIMENT_SEEDS})")
    print("="*80)
    print(" All models will be saved for comprehensive analysis")
    print(" Naming convention: model_{model_num}_seed_{seed_idx}.keras")
    print(" Using stratified 80/20 train-test split for each seed")
    print("="*80)

    for seed_idx, current_seed in enumerate(EXPERIMENT_SEEDS):
        print(f"\n>>> RUN {seed_idx+1}/{len(EXPERIMENT_SEEDS)} | Random Seed:␣
        ↪{current_seed}")

        # Reset random states for this run

```

```

np.random.seed(current_seed)
random.seed(current_seed)
tf.random.set_seed(current_seed)

# Stratified train-test split with current seed
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=TEST_SIZE,
    stratify=y,
    random_state=current_seed
)

# STORE THE TEST SET FOR THIS SEED (for later evaluation)
test_sets[seed_idx + 1] = (X_test, y_test)

print(f"  Train: {len(X_train)} samples | Test: {len(X_test)} samples")

# Calculate class weights
class_weights_train = compute_class_weight('balanced', classes=np.
↪unique(y_train), y=y_train)
class_weight_dict = {0: class_weights_train[0], 1: class_weights_train[1]}

# --- Augmentation Logic ---
X_train_fresh = X_train[y_train == 0]
y_train_fresh = y_train[y_train == 0]
X_train_infected = X_train[y_train == 1]
y_train_infected = y_train[y_train == 1]

target_fresh = len(X_train_fresh) * AUGMENTATIONS_PER_IMAGE
target_infected = len(X_train_infected) * AUGMENTATIONS_PER_IMAGE

X_aug_fresh, y_aug_fresh = generate_augmented_images(X_train_fresh, ↪
↪y_train_fresh, target_fresh, "Fresh")
X_aug_infected, y_aug_infected = ↪
↪generate_augmented_images(X_train_infected, y_train_infected, ↪
↪target_infected, "Infected")

X_train_augmented = np.concatenate([X_aug_fresh, X_aug_infected], axis=0)
y_train_augmented = np.concatenate([y_aug_fresh, y_aug_infected], axis=0)

shuffle_indices = np.random.permutation(len(X_train_augmented))
X_train_augmented = X_train_augmented[shuffle_indices]
y_train_augmented = y_train_augmented[shuffle_indices]

# Normalize
X_train_augmented = X_train_augmented / 255.0
X_test_scaled = X_test / 255.0

```

```

print(f" Augmented training set: {len(X_train_augmented)} samples")

# Storage for this seed's predictions
seed_predictions = []
seed_probabilities = []

# Train each model
for model_name, num_layers in model_configs.items():
    print(f" Training {model_name}...", end="\r")

    # Create fresh model
    model = create_cnn_model(num_layers)

    try:
        history = model.fit(
            X_train_augmented, y_train_augmented,
            batch_size=BATCH_SIZE,
            epochs=EPOCHS,
            validation_data=(X_test_scaled, y_test),
            callbacks=callbacks,
            class_weight=class_weight_dict,
            verbose=0
        )
    except Exception:
        # Retry with smaller batch size
        K.clear_session()
        gc.collect()
        model = create_cnn_model(num_layers)
        history = model.fit(
            X_train_augmented, y_train_augmented,
            batch_size=8,
            epochs=EPOCHS,
            validation_data=(X_test_scaled, y_test),
            callbacks=callbacks,
            class_weight=class_weight_dict,
            verbose=0
        )

    # Predict on test set
    y_pred_proba = model.predict(X_test_scaled, verbose=0)
    y_pred = (y_pred_proba > 0.5).astype(int).flatten()

    # Store for ensemble
    seed_predictions.append(y_pred)
    seed_probabilities.append(y_pred_proba.flatten())

```



```

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    multi_run_results.append({
        'Seed': current_seed,
        'SeedIdx': seed_idx + 1,
        'Model': model_name,
        'Accuracy': acc,
        'Recall': rec,
        'Precision': prec,
        'F1': f1
    })

    # Save model
    model_num = model_name.split('_')[1]
    save_path = f'models/model_{model_num}_seed_{seed_idx+1}.keras'
    model.save(save_path)
    saved_model_paths[(model_name, seed_idx+1)] = save_path

    print(f"    {model_name}: Acc={acc:.4f}, Rec={rec:.4f}, Prec={prec:.4f}, F1={f1:.4f}")

    K.clear_session()
    gc.collect()

    # Calculate Ensemble for this seed
    seed_probabilities = np.array(seed_probabilities)
    ensemble_proba = np.mean(seed_probabilities, axis=0) # Soft voting
    ensemble_pred = (ensemble_proba > 0.5).astype(int)

    # Ensemble metrics
    e_acc = accuracy_score(y_test, ensemble_pred)
    e_rec = recall_score(y_test, ensemble_pred)
    e_prec = precision_score(y_test, ensemble_pred)
    e_f1 = f1_score(y_test, ensemble_pred)
    e_auc = roc_auc_score(y_test, ensemble_proba)
    e_cm = confusion_matrix(y_test, ensemble_pred)
    e_tn, e_fp, e_fn, e_tp = e_cm.ravel()
    e_spec = e_tn / (e_tn + e_fp) if (e_tn + e_fp) > 0 else 0

    multi_run_results.append({
        'Seed': current_seed,
        'SeedIdx': seed_idx + 1,
        'Model': 'Ensemble',

```

```

        'Accuracy': e_acc,
        'Recall': e_rec,
        'Precision': e_prec,
        'F1': e_f1,
        'AUC-ROC': e_auc,
        'Specificity': e_spec,
        'TP': e_tp,
        'TN': e_tn,
        'FP': e_fp,
        'FN': e_fn
    })

    print(f"    Ensemble: Acc={e_acc:.4f}, Rec={e_rec:.4f}, Prec={e_prec:.4f}, F1={e_f1:.4f}")

# --- Final Statistical Summary ---
print("\n" + "="*80)
print("MULTI-SEED EXPERIMENT COMPLETED")
print("="*80)

stats_df = pd.DataFrame(multi_run_results)
summary = stats_df.groupby('Model').agg(['mean', 'std'])[['Accuracy', 'Recall', 'Precision', 'F1']]

# Flatten columns for display
summary.columns = ['_'.join(col).strip() for col in summary.columns.values]
summary = summary.reset_index()

print(f"\nModel Performance (Mean ± Std Dev over {len(EXPERIMENT_SEEDS)} seeds):")
print(summary.to_string())

# Save statistical results
summary.to_csv('multi_seed_statistics.csv', index=False)

# Print saved model paths
print("\n" + "="*80)
print("SAVED MODELS SUMMARY")
print("="*80)
for (model_name, seed_num), path in sorted(saved_model_paths.items()):
    print(f"    {model_name} Seed {seed_num}: {path}")

print("\n    Note: Run the next cell to compute seed-averaged predictions")
print("    This will rebuild evaluation_results with TRUE average performance.")

```

=====

STARTING MULTI-SEED EXPERIMENT (Seeds: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29])

```
=====
All models will be saved for comprehensive analysis
Naming convention: model_{model_num}_seed_{seed_idx}.keras
Using stratified 80/20 train-test split for each seed
=====
```

>>> RUN 1/30 | Random Seed: 0

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730

Model_2: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744

Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 2/30 | Random Seed: 1

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730

WARNING:tensorflow:5 out of the last 5 calls to <function Model.make_predict_function.<locals>.predict_function at 0x000001BE39659C60> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to

https://www.tensorflow.org/guide/function#controlling_retracing and

https://www.tensorflow.org/api_docs/python/tf/function for more details.

Model_2: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744

WARNING:tensorflow:6 out of the last 6 calls to <function Model.make_predict_function.<locals>.predict_function at 0x000001BE3965A320> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to

https://www.tensorflow.org/guide/function#controlling_retracing and

https://www.tensorflow.org/api_docs/python/tf/function for more details.

Model_3: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474

Ensemble: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744

>>> RUN 3/30 | Random Seed: 2

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

Model_2: Acc=0.8621, Rec=1.0000, Prec=0.8261, F1=0.9048
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 4/30 | Random Seed: 3

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 5/30 | Random Seed: 4

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=0.9310, Rec=1.0000, Prec=0.9048, F1=0.9500
Model_3: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 6/30 | Random Seed: 5

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 7/30 | Random Seed: 6

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730
Model_2: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 8/30 | Random Seed: 7

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.8966, Rec=1.0000, Prec=0.8636, F1=0.9268
Model_2: Acc=0.8621, Rec=1.0000, Prec=0.8261, F1=0.9048
Model_3: Acc=0.9310, Rec=1.0000, Prec=0.9048, F1=0.9500
Ensemble: Acc=0.9310, Rec=1.0000, Prec=0.9048, F1=0.9500

>>> RUN 9/30 | Random Seed: 8

Train: 112 samples | Test: 29 samples

Augmented training set: 1120 samples

Model_1: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730

Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.9310, Rec=0.8947, Prec=1.0000, F1=0.9444
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 10/30 | Random Seed: 9
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Model_2: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474
Model_3: Acc=0.9310, Rec=0.8947, Prec=1.0000, F1=0.9444
Ensemble: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474

>>> RUN 11/30 | Random Seed: 10
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.8966, Rec=0.9474, Prec=0.9000, F1=0.9231
Model_2: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Model_3: Acc=0.8966, Rec=0.8947, Prec=0.9444, F1=0.9189
Ensemble: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474

>>> RUN 12/30 | Random Seed: 11
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 13/30 | Random Seed: 12
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 14/30 | Random Seed: 13
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 15/30 | Random Seed: 14
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.9655, Rec=0.9474, Prec=1.0000, F1=0.9730

Model_2: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474
Model_3: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Ensemble: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744

>>> RUN 16/30 | Random Seed: 15
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 17/30 | Random Seed: 16
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=0.7931, Rec=0.7368, Prec=0.9333, F1=0.8235
Model_3: Acc=0.8621, Rec=0.7895, Prec=1.0000, F1=0.8824
Ensemble: Acc=0.9310, Rec=0.8947, Prec=1.0000, F1=0.9444

>>> RUN 18/30 | Random Seed: 17
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=0.8276, Rec=1.0000, Prec=0.7917, F1=0.8837
Model_3: Acc=0.9310, Rec=0.8947, Prec=1.0000, F1=0.9444
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 19/30 | Random Seed: 18
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 20/30 | Random Seed: 19
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.9310, Rec=0.9474, Prec=0.9474, F1=0.9474
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 21/30 | Random Seed: 20
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.8966, Rec=0.8421, Prec=1.0000, F1=0.9143

Model_2: Acc=0.8621, Rec=1.0000, Prec=0.8261, F1=0.9048
Model_3: Acc=0.8276, Rec=0.8421, Prec=0.8889, F1=0.8649
Ensemble: Acc=0.8966, Rec=0.9474, Prec=0.9000, F1=0.9231

>>> RUN 22/30 | Random Seed: 21
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=0.8276, Rec=1.0000, Prec=0.7917, F1=0.8837
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 23/30 | Random Seed: 22
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.6552, Rec=0.4737, Prec=1.0000, F1=0.6429
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 24/30 | Random Seed: 23
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.8966, Rec=0.8947, Prec=0.9444, F1=0.9189
Model_2: Acc=0.8966, Rec=0.8421, Prec=1.0000, F1=0.9143
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=0.9310, Rec=0.8947, Prec=1.0000, F1=0.9444

>>> RUN 25/30 | Random Seed: 24
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

>>> RUN 26/30 | Random Seed: 25
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=0.6207, Rec=0.4737, Prec=0.9000, F1=0.6207
Model_2: Acc=0.8966, Rec=0.8947, Prec=0.9444, F1=0.9189
Model_3: Acc=0.7241, Rec=0.6316, Prec=0.9231, F1=0.7500
Ensemble: Acc=0.7586, Rec=0.6842, Prec=0.9286, F1=0.7879

>>> RUN 27/30 | Random Seed: 26
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
 Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
 Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000

```
>>> RUN 28/30 | Random Seed: 27
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
Model_3: Acc=0.9310, Rec=1.0000, Prec=0.9048, F1=0.9500
Ensemble: Acc=0.9655, Rec=1.0000, Prec=0.9500, F1=0.9744
```

```
>>> RUN 29/30 | Random Seed: 28
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=0.6207, Rec=0.8947, Prec=0.6538, F1=0.7556
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
```

```
>>> RUN 30/30 | Random Seed: 29
Train: 112 samples | Test: 29 samples
Augmented training set: 1120 samples
Model_1: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_2: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Model_3: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
Ensemble: Acc=1.0000, Rec=1.0000, Prec=1.0000, F1=1.0000
```

```
=====
MULTI-SEED EXPERIMENT COMPLETED
=====
```

Model Performance (Mean $\pm$ Std Dev over 30 seeds):

	Model	Accuracy_mean	Accuracy_std	Recall_mean	Recall_std
	Precision_mean	Precision_std	F1_mean	F1_std	
0	Ensemble	0.973563	0.050972	0.977193	0.062860
		0.982602	0.031146	0.978922	0.043059
1	Model_1	0.965517	0.074123	0.963158	0.099814
		0.983603	0.036491	0.970579	0.071455
2	Model_2	0.948276	0.064511	0.978947	0.057985
		0.949629	0.068498	0.961833	0.048045
3	Model_3	0.932184	0.101440	0.935088	0.120244
		0.965484	0.068635	0.944622	0.086897

```
=====
SAVED MODELS SUMMARY
=====
Model_1 Seed 1: models/model_1_seed_1.keras
```


Model_1 Seed 2: models/model_1_seed_2.keras
Model_1 Seed 3: models/model_1_seed_3.keras
Model_1 Seed 4: models/model_1_seed_4.keras
Model_1 Seed 5: models/model_1_seed_5.keras
Model_1 Seed 6: models/model_1_seed_6.keras
Model_1 Seed 7: models/model_1_seed_7.keras
Model_1 Seed 8: models/model_1_seed_8.keras
Model_1 Seed 9: models/model_1_seed_9.keras
Model_1 Seed 10: models/model_1_seed_10.keras
Model_1 Seed 11: models/model_1_seed_11.keras
Model_1 Seed 12: models/model_1_seed_12.keras
Model_1 Seed 13: models/model_1_seed_13.keras
Model_1 Seed 14: models/model_1_seed_14.keras
Model_1 Seed 15: models/model_1_seed_15.keras
Model_1 Seed 16: models/model_1_seed_16.keras
Model_1 Seed 17: models/model_1_seed_17.keras
Model_1 Seed 18: models/model_1_seed_18.keras
Model_1 Seed 19: models/model_1_seed_19.keras
Model_1 Seed 20: models/model_1_seed_20.keras
Model_1 Seed 21: models/model_1_seed_21.keras
Model_1 Seed 22: models/model_1_seed_22.keras
Model_1 Seed 23: models/model_1_seed_23.keras
Model_1 Seed 24: models/model_1_seed_24.keras
Model_1 Seed 25: models/model_1_seed_25.keras
Model_1 Seed 26: models/model_1_seed_26.keras
Model_1 Seed 27: models/model_1_seed_27.keras
Model_1 Seed 28: models/model_1_seed_28.keras
Model_1 Seed 29: models/model_1_seed_29.keras
Model_1 Seed 30: models/model_1_seed_30.keras
Model_2 Seed 1: models/model_2_seed_1.keras
Model_2 Seed 2: models/model_2_seed_2.keras
Model_2 Seed 3: models/model_2_seed_3.keras
Model_2 Seed 4: models/model_2_seed_4.keras
Model_2 Seed 5: models/model_2_seed_5.keras
Model_2 Seed 6: models/model_2_seed_6.keras
Model_2 Seed 7: models/model_2_seed_7.keras
Model_2 Seed 8: models/model_2_seed_8.keras
Model_2 Seed 9: models/model_2_seed_9.keras
Model_2 Seed 10: models/model_2_seed_10.keras
Model_2 Seed 11: models/model_2_seed_11.keras
Model_2 Seed 12: models/model_2_seed_12.keras
Model_2 Seed 13: models/model_2_seed_13.keras
Model_2 Seed 14: models/model_2_seed_14.keras
Model_2 Seed 15: models/model_2_seed_15.keras
Model_2 Seed 16: models/model_2_seed_16.keras
Model_2 Seed 17: models/model_2_seed_17.keras
Model_2 Seed 18: models/model_2_seed_18.keras
Model_2 Seed 19: models/model_2_seed_19.keras

Model_2 Seed 20: models/model_2_seed_20.keras
Model_2 Seed 21: models/model_2_seed_21.keras
Model_2 Seed 22: models/model_2_seed_22.keras
Model_2 Seed 23: models/model_2_seed_23.keras
Model_2 Seed 24: models/model_2_seed_24.keras
Model_2 Seed 25: models/model_2_seed_25.keras
Model_2 Seed 26: models/model_2_seed_26.keras
Model_2 Seed 27: models/model_2_seed_27.keras
Model_2 Seed 28: models/model_2_seed_28.keras
Model_2 Seed 29: models/model_2_seed_29.keras
Model_2 Seed 30: models/model_2_seed_30.keras
Model_3 Seed 1: models/model_3_seed_1.keras
Model_3 Seed 2: models/model_3_seed_2.keras
Model_3 Seed 3: models/model_3_seed_3.keras
Model_3 Seed 4: models/model_3_seed_4.keras
Model_3 Seed 5: models/model_3_seed_5.keras
Model_3 Seed 6: models/model_3_seed_6.keras
Model_3 Seed 7: models/model_3_seed_7.keras
Model_3 Seed 8: models/model_3_seed_8.keras
Model_3 Seed 9: models/model_3_seed_9.keras
Model_3 Seed 10: models/model_3_seed_10.keras
Model_3 Seed 11: models/model_3_seed_11.keras
Model_3 Seed 12: models/model_3_seed_12.keras
Model_3 Seed 13: models/model_3_seed_13.keras
Model_3 Seed 14: models/model_3_seed_14.keras
Model_3 Seed 15: models/model_3_seed_15.keras
Model_3 Seed 16: models/model_3_seed_16.keras
Model_3 Seed 17: models/model_3_seed_17.keras
Model_3 Seed 18: models/model_3_seed_18.keras
Model_3 Seed 19: models/model_3_seed_19.keras
Model_3 Seed 20: models/model_3_seed_20.keras
Model_3 Seed 21: models/model_3_seed_21.keras
Model_3 Seed 22: models/model_3_seed_22.keras
Model_3 Seed 23: models/model_3_seed_23.keras
Model_3 Seed 24: models/model_3_seed_24.keras
Model_3 Seed 25: models/model_3_seed_25.keras
Model_3 Seed 26: models/model_3_seed_26.keras
Model_3 Seed 27: models/model_3_seed_27.keras
Model_3 Seed 28: models/model_3_seed_28.keras
Model_3 Seed 29: models/model_3_seed_29.keras
Model_3 Seed 30: models/model_3_seed_30.keras

Note: Run the next cell to compute seed-averaged predictions
This will rebuild evaluation_results with TRUE average performance.

1.2 5.4 Compute Mean Metrics Across Seeds

This cell computes the **mean of metrics** (not averaged predictions) across all 5 seeds for each model architecture. For each model:

1. Load all saved models (e.g., `model_1_seed_1.keras`, ..., `model_1_seed_5.keras`)
2. Calculate **full metrics** for each seed (Accuracy, Recall, Precision, F1, AUC, Specificity, TP, TN, FP, FN)
3. Take the **mean and standard deviation** of those metrics
4. Select a **representative seed** (closest to mean Recall) for visualizations requiring actual predictions

This approach gives the true average performance and uncertainty bounds, rather than creating a pseudo-ensemble from averaged predictions.

1.3 5.1 Load Saved Models (Skip Training)

Run this cell if you want to skip training and load pre-trained models.

This cell will: 1. Load all required libraries and configurations 2. Load the dataset 3. Recreate the test sets using the same random seeds 4. Initialize the data structures needed for evaluation

Note: Only run this cell if you have NOT run the training cells above. If you just ran training, skip this cell.

```
[18]: # =====  
# SKIP TRAINING: LOAD PRE-TRAINED MODELS  
# =====  
# Run this cell ONLY if you want to skip training and use saved models.  
# This cell sets up all required variables for the evaluation cells below.  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from pathlib import Path  
import os  
import random  
import gc  
import glob  
  
# Deep Learning libraries  
import tensorflow as tf  
from tensorflow import keras  
from tensorflow.keras import layers, models  
from tensorflow.keras import backend as K  
from tensorflow.keras.preprocessing.image import load_img, img_to_array  
  
# Scikit-learn libraries  
from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)

# =====
# HYPERPARAMETERS (must match training)
# =====
SEED = 42
EXPERIMENT_SEEDS = [i for i in range(30)] # 30 seeds for statistical analysis

IMG_WIDTH = 400
IMG_HEIGHT = 166
IMG_CHANNELS = 3
TEST_SIZE = 0.2

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

# =====
# FIGURE STYLING (for report-quality plots)
# =====
BASE_FONT = {
    'title': 24, 'suptitle': 28, 'label': 22,
    'tick': 20, 'legend': 18, 'annotation': 18,
}

def get_font_scale(n_rows, n_cols):
    n_panels = n_rows * n_cols
    if n_panels == 1: return 1.0
    elif n_panels == 2: return 1.3
    elif n_panels <= 4: return 1.5
    elif n_panels <= 6: return 1.7
    elif n_panels <= 9: return 1.8
    else: return 2.0

def get_fonts(n_rows=1, n_cols=1):
    scale = get_font_scale(n_rows, n_cols)
    return {k: int(v * scale) for k, v in BASE_FONT.items()}

plt.rcParams.update({
    'figure.facecolor': 'white', 'axes.facecolor': 'white',
    'axes.grid': True, 'grid.alpha': 0.3,
    'axes.spines.top': False, 'axes.spines.right': False,
    'figure.dpi': 100, 'savefig.dpi': 300,
    'savefig.bbox': 'tight', 'savefig.facecolor': 'white',
    'font.family': 'sans-serif',

```

```

})

Path('report/figures').mkdir(parents=True, exist_ok=True)

# =====
# GPU CONFIGURATION
# =====
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f" GPU available: {gpus[0].name}")
        from tensorflow.keras import mixed_precision
        mixed_precision.set_global_policy('mixed_float16')
        print(" Mixed precision (FP16) enabled")
    except RuntimeError as e:
        print(f"GPU configuration error: {e}")
else:
    print("No GPU found, using CPU")

# =====
# LOAD DATASET
# =====
DATA_DIR = Path('Data-SalmonScan')
FRESH_DIR = DATA_DIR / 'FreshFish'
INFECTED_DIR = DATA_DIR / 'InfectedFish'

def load_dataset(fresh_dir, infected_dir):
    """Load all images and labels from directories."""
    images, labels, filenames = [], [], []

    # Load fresh fish images (label = 0)
    for img_path in sorted(list(fresh_dir.glob('*.png'))):
        try:
            img = load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            images.append(img_to_array(img))
            labels.append(0)
            filenames.append(img_path.name)
        except Exception as e:
            print(f"Error loading {img_path}: {e}")

    # Load infected fish images (label = 1)
    for img_path in sorted(list(infected_dir.glob('*.png'))):
        try:
            img = load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            images.append(img_to_array(img))

```

```

        labels.append(1)
        filenames.append(img_path.name)
    except Exception as e:
        print(f"Error loading {img_path}: {e}")

    return np.array(images), np.array(labels), filenames

X, y, filenames = load_dataset(FRESH_DIR, INFECTED_DIR)
print(f"\n Dataset loaded: {len(X)} images")
print(f"  Fresh: {sum(y==0)}, Infected: {sum(y==1)}")
print(f"  Image shape: {X[0].shape}")

# =====
# PREPARE TEST SETS FOR ALL SEEDS
# =====
test_sets = {}
for seed_idx in range(1, len(EXPERIMENT_SEEDS) + 1):
    _, X_test_for_seed, _, y_test_for_seed = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=EXPERIMENT_SEEDS[seed_idx - 1],
        stratify=y
    )
    test_sets[seed_idx] = (X_test_for_seed, y_test_for_seed)

print(f"\n Test sets prepared for {len(EXPERIMENT_SEEDS)} seeds")

# =====
# INITIALIZE seed_metrics DICTIONARY FOR MULTI-SEED ANALYSIS
# =====
seed_metrics = {'Model_1': [], 'Model_2': [], 'Model_3': []}
multi_run_results = []

print("\n" + "="*70)
print("LOADING PRE-TRAINED MODELS")
print("="*70)

# Process all model files
model_files = sorted(glob.glob('models/model_*.keras'))
print(f"Found {len(model_files)} model files")

for model_file in model_files:
    parts = os.path.basename(model_file).replace('.keras', '').split('_')
    model_num = int(parts[1])
    current_seed = int(parts[3])
    model_name = f'Model_{model_num}'

    try:
        loaded_model = models.load_model(model_file)

```

```

X_test_seed, y_test_seed = test_sets[current_seed]
X_test_seed_scaled = X_test_seed / 255.0

y_proba = loaded_model.predict(X_test_seed_scaled, verbose=0).flatten()
y_pred_seed = (y_proba > 0.5).astype(int)

acc = accuracy_score(y_test_seed, y_pred_seed)
rec = recall_score(y_test_seed, y_pred_seed)
prec = precision_score(y_test_seed, y_pred_seed)
f1 = f1_score(y_test_seed, y_pred_seed)
auc = roc_auc_score(y_test_seed, y_proba)
cm = confusion_matrix(y_test_seed, y_pred_seed)
tn, fp, fn, tp = cm.ravel()
spec = tn / (tn + fp) if (tn + fp) > 0 else 0

seed_metrics[model_name].append({
    'seed_idx': current_seed,
    'Accuracy': acc, 'Recall': rec, 'Precision': prec,
    'F1-Score': f1, 'AUC-ROC': auc, 'Specificity': spec,
    'True Positives': tp, 'True Negatives': tn,
    'False Positives': fp, 'False Negatives': fn,
    'y_test': y_test_seed, 'Predictions': y_pred_seed, 'Probabilities': y_proba
})

K.clear_session()
gc.collect()

except Exception as e:
    print(f"Error loading {model_file}: {e}")

print("\n All models loaded and evaluated")
for model_name in seed_metrics:
    print(f" {model_name}: {len(seed_metrics[model_name])} seeds")

```

GPU available: /physical_device:GPU:0
Mixed precision (FP16) enabled

Dataset loaded: 141 images
Fresh: 50, Infected: 91
Image shape: (166, 400, 3)

Test sets prepared for 30 seeds

```

=====
LOADING PRE-TRAINED MODELS
=====
Found 90 model files

```

All models loaded and evaluated
Model_1: 30 seeds
Model_2: 30 seeds
Model_3: 30 seeds

```
[3]: # =====  
# COMPUTE MEAN METRICS ACROSS SEEDS (NOT AVERAGED PREDICTIONS)  
# =====  
# This cell loads all saved models, calculates FULL metrics for each seed,  
# then takes the MEAN of those metrics. This gives the true average performance  
# rather than creating a pseudo-ensemble from averaged predictions.  
  
import glob  
  
print("="*100)  
print("  COMPUTING MEAN METRICS ACROSS SEEDS")  
print("="*100)  
  
# =====  
# VERIFY TEST SETS ARE AVAILABLE  
# =====  
if 'test_sets' not in dir() or len(test_sets) == 0:  
    print("\n  WARNING: test_sets not found! Re-running train-test splits...")  
    test_sets = {}  
    for seed_idx, current_seed in enumerate(EXPERIMENT_SEEDS):  
        np.random.seed(current_seed)  
        _, X_test, _, y_test = train_test_split(  
            X, y,  
            test_size=TEST_SIZE,  
            stratify=y,  
            random_state=current_seed  
        )  
        test_sets[seed_idx + 1] = (X_test, y_test)  
    print(f"  Recreated test sets for {len(test_sets)} seeds")  
  
print(f"\n  Test sets available for {len(test_sets)} seeds:")  
for seed_idx, (X_test, y_test) in test_sets.items():  
    print(f"    Seed {seed_idx}: {len(X_test)} test samples (Fresh:   
    ↪{sum(y_test==0)}, Infected: {sum(y_test==1)})")  
  
# Find all saved models  
model_files = sorted(glob.glob('models/model_*_seed_*.keras'))  
print(f"\n  Found {len(model_files)} saved models:")  
for f in model_files:  
    print(f"    {f}")
```



```

# Storage for FULL metrics per model per seed
# Structure: {model_name: [{seed_idx: 1, metrics...}, {seed_idx: 2, metrics...
↪}, ...]}
seed_metrics = {
    'Model_1': [],
    'Model_2': [],
    'Model_3': []
}

# Also store predictions from representative seed for visualizations
representative_predictions = {}

# Load each model and calculate FULL metrics
print("\n" + "-"*100)
print("Loading models and computing per-seed metrics...")
print("-"*100)

for model_file in model_files:
    # Parse filename to get model and seed
    filename = os.path.basename(model_file)
    parts = filename.replace('.keras', '').split('_')
    model_num = parts[1]
    seed_idx = int(parts[3])
    model_name = f"Model_{model_num}"

    print(f"\n    {filename} (evaluating on seed {seed_idx} test set:␣
↪{len(test_sets[seed_idx][0])} samples)...")

    try:
        loaded_model = models.load_model(model_file)

        # GET THE CORRECT TEST SET FOR THIS SEED
        X_test_for_seed, y_test_for_seed = test_sets[seed_idx]
        X_test_scaled = X_test_for_seed / 255.0

        proba = loaded_model.predict(X_test_scaled, verbose=0).flatten()
        pred = (proba > 0.5).astype(int)

        # Calculate FULL metrics for this seed (using correct test labels)
        acc = accuracy_score(y_test_for_seed, pred)
        rec = recall_score(y_test_for_seed, pred)
        prec = precision_score(y_test_for_seed, pred)
        f1 = f1_score(y_test_for_seed, pred)
        auc = roc_auc_score(y_test_for_seed, proba)

        # Confusion matrix values
        cm = confusion_matrix(y_test_for_seed, pred)

```

```

tn, fp, fn, tp = cm.ravel()
spec = tn / (tn + fp) if (tn + fp) > 0 else 0

# Store all metrics for this seed
seed_metrics[model_name].append({
    'seed_idx': seed_idx,
    'Accuracy': acc,
    'Recall': rec,
    'Precision': prec,
    'F1-Score': f1,
    'AUC-ROC': auc,
    'Specificity': spec,
    'True Positives': tp,
    'True Negatives': tn,
    'False Positives': fp,
    'False Negatives': fn,
    'Predictions': pred,
    'Probabilities': proba,
    'Confusion Matrix': cm,
    'y_test': y_test_for_seed
})

print(f"      Acc: {acc:.4f} | Rec: {rec:.4f} | Prec: {prec:.4f} | F1: {f1:.4f} | AUC: {auc:.4f}")
print(f"      TP: {tp} | TN: {tn} | FP: {fp} | FN: {fn}")

K.clear_session()
gc.collect()
except Exception as e:
    print(f"      Error: {e}")

# =====
# CALCULATE MEAN METRICS AND SELECT REPRESENTATIVE SEED
# =====
print("\n" + "="*100)
print("  CALCULATING MEAN METRICS ACROSS SEEDS")
print("="*100)

# Clear old evaluation_results
evaluation_results = []

# Metrics to average (including confusion matrix values)
metrics_to_average = ['Accuracy', 'Recall', 'Precision', 'F1-Score', 'AUC-ROC',
    'Specificity',
    'True Positives', 'True Negatives', 'False Positives',
    'False Negatives']

```

```

for model_name in ['Model_1', 'Model_2', 'Model_3']:
    if seed_metrics[model_name]:
        n_seeds = len(seed_metrics[model_name])

        print(f"\n{'='*70}")
        print(f"{model_name} - Statistics across {n_seeds} seeds")
        print(f"{'='*70}")

        # Calculate mean and std for each metric
        mean_metrics = {}
        std_metrics = {}
        for metric in metrics_to_average:
            values = [s[metric] for s in seed_metrics[model_name]]
            mean_metrics[metric] = np.mean(values)
            std_metrics[metric] = np.std(values)
            print(f"    {metric}: {mean_metrics[metric]:.4f} ±
↪{std_metrics[metric]:.4f}")

            # Select representative seed (closest to mean Recall)
            mean_recall = mean_metrics['Recall']
            representative_seed_data = min(seed_metrics[model_name],
                                           key=lambda x: abs(x['Recall'] -
↪mean_recall))
            representative_seed_idx = representative_seed_data['seed_idx']

            print(f"\n    Representative Seed: {representative_seed_idx} (Recall
↪closest to mean)")

            # Store in evaluation_results for downstream cells
            evaluation_results.append({
                'Model': model_name,
                'Accuracy': mean_metrics['Accuracy'],
                'Accuracy_std': std_metrics['Accuracy'],
                'Recall': mean_metrics['Recall'],
                'Recall_std': std_metrics['Recall'],
                'Precision': mean_metrics['Precision'],
                'Precision_std': std_metrics['Precision'],
                'F1-Score': mean_metrics['F1-Score'],
                'F1-Score_std': std_metrics['F1-Score'],
                'AUC-ROC': mean_metrics['AUC-ROC'],
                'AUC-ROC_std': std_metrics['AUC-ROC'],
                'Specificity': mean_metrics['Specificity'],
                'Specificity_std': std_metrics['Specificity'],
                'True Positives': mean_metrics['True Positives'],
                'True Negatives': mean_metrics['True Negatives'],
                'False Positives': mean_metrics['False Positives'],
                'False Negatives': mean_metrics['False Negatives'],

```

```

        'Representative_Seed': representative_seed_idx,
        'Predictions': representative_seed_data['Predictions'],
        'Probabilities': representative_seed_data['Probabilities'],
        'Confusion Matrix': representative_seed_data['Confusion Matrix'],
        'y_test': representative_seed_data['y_test']
    })

print("\n" + "="*100)
print("  EVALUATION COMPLETE")
print("="*100)
print(f"\nevaluation_results populated with {len(evaluation_results)} models")
for r in evaluation_results:
    print(f"    {r['Model']}: Acc={r['Accuracy']:.4f}±{r['Accuracy_std']:.4f},
    ↳Rep.Seed={r['Representative_Seed']}")

```

```

=====
=====
COMPUTING MEAN METRICS ACROSS SEEDS
=====
=====

```

Test sets available for 30 seeds:

```

Seed 1: 29 test samples (Fresh: 10, Infected: 19)
Seed 2: 29 test samples (Fresh: 10, Infected: 19)
Seed 3: 29 test samples (Fresh: 10, Infected: 19)
Seed 4: 29 test samples (Fresh: 10, Infected: 19)
Seed 5: 29 test samples (Fresh: 10, Infected: 19)
Seed 6: 29 test samples (Fresh: 10, Infected: 19)
Seed 7: 29 test samples (Fresh: 10, Infected: 19)
Seed 8: 29 test samples (Fresh: 10, Infected: 19)
Seed 9: 29 test samples (Fresh: 10, Infected: 19)
Seed 10: 29 test samples (Fresh: 10, Infected: 19)
Seed 11: 29 test samples (Fresh: 10, Infected: 19)
Seed 12: 29 test samples (Fresh: 10, Infected: 19)
Seed 13: 29 test samples (Fresh: 10, Infected: 19)
Seed 14: 29 test samples (Fresh: 10, Infected: 19)
Seed 15: 29 test samples (Fresh: 10, Infected: 19)
Seed 16: 29 test samples (Fresh: 10, Infected: 19)
Seed 17: 29 test samples (Fresh: 10, Infected: 19)
Seed 18: 29 test samples (Fresh: 10, Infected: 19)
Seed 19: 29 test samples (Fresh: 10, Infected: 19)
Seed 20: 29 test samples (Fresh: 10, Infected: 19)
Seed 21: 29 test samples (Fresh: 10, Infected: 19)
Seed 22: 29 test samples (Fresh: 10, Infected: 19)
Seed 23: 29 test samples (Fresh: 10, Infected: 19)
Seed 24: 29 test samples (Fresh: 10, Infected: 19)
Seed 25: 29 test samples (Fresh: 10, Infected: 19)
Seed 26: 29 test samples (Fresh: 10, Infected: 19)

```

Seed 27: 29 test samples (Fresh: 10, Infected: 19)
Seed 28: 29 test samples (Fresh: 10, Infected: 19)
Seed 29: 29 test samples (Fresh: 10, Infected: 19)
Seed 30: 29 test samples (Fresh: 10, Infected: 19)

Found 90 saved models:

models\model_1_seed_1.keras
models\model_1_seed_10.keras
models\model_1_seed_11.keras
models\model_1_seed_12.keras
models\model_1_seed_13.keras
models\model_1_seed_14.keras
models\model_1_seed_15.keras
models\model_1_seed_16.keras
models\model_1_seed_17.keras
models\model_1_seed_18.keras
models\model_1_seed_19.keras
models\model_1_seed_2.keras
models\model_1_seed_20.keras
models\model_1_seed_21.keras
models\model_1_seed_22.keras
models\model_1_seed_23.keras
models\model_1_seed_24.keras
models\model_1_seed_25.keras
models\model_1_seed_26.keras
models\model_1_seed_27.keras
models\model_1_seed_28.keras
models\model_1_seed_29.keras
models\model_1_seed_3.keras
models\model_1_seed_30.keras
models\model_1_seed_4.keras
models\model_1_seed_5.keras
models\model_1_seed_6.keras
models\model_1_seed_7.keras
models\model_1_seed_8.keras
models\model_1_seed_9.keras
models\model_2_seed_1.keras
models\model_2_seed_10.keras
models\model_2_seed_11.keras
models\model_2_seed_12.keras
models\model_2_seed_13.keras
models\model_2_seed_14.keras
models\model_2_seed_15.keras
models\model_2_seed_16.keras
models\model_2_seed_17.keras
models\model_2_seed_18.keras
models\model_2_seed_19.keras
models\model_2_seed_2.keras

models\model_2_seed_20.keras
models\model_2_seed_21.keras
models\model_2_seed_22.keras
models\model_2_seed_23.keras
models\model_2_seed_24.keras
models\model_2_seed_25.keras
models\model_2_seed_26.keras
models\model_2_seed_27.keras
models\model_2_seed_28.keras
models\model_2_seed_29.keras
models\model_2_seed_3.keras
models\model_2_seed_30.keras
models\model_2_seed_4.keras
models\model_2_seed_5.keras
models\model_2_seed_6.keras
models\model_2_seed_7.keras
models\model_2_seed_8.keras
models\model_2_seed_9.keras
models\model_3_seed_1.keras
models\model_3_seed_10.keras
models\model_3_seed_11.keras
models\model_3_seed_12.keras
models\model_3_seed_13.keras
models\model_3_seed_14.keras
models\model_3_seed_15.keras
models\model_3_seed_16.keras
models\model_3_seed_17.keras
models\model_3_seed_18.keras
models\model_3_seed_19.keras
models\model_3_seed_2.keras
models\model_3_seed_20.keras
models\model_3_seed_21.keras
models\model_3_seed_22.keras
models\model_3_seed_23.keras
models\model_3_seed_24.keras
models\model_3_seed_25.keras
models\model_3_seed_26.keras
models\model_3_seed_27.keras
models\model_3_seed_28.keras
models\model_3_seed_29.keras
models\model_3_seed_3.keras
models\model_3_seed_30.keras
models\model_3_seed_4.keras
models\model_3_seed_5.keras
models\model_3_seed_6.keras
models\model_3_seed_7.keras
models\model_3_seed_8.keras
models\model_3_seed_9.keras

Loading models and computing per-seed metrics..

model_1_seed_1.keras (evaluating on seed 1 test set: 29 samples)...
Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 0.9947
TP: 18 | TN: 10 | FP: 0 | FN: 1

model_1_seed_10.keras (evaluating on seed 10 test set: 29 samples)...
Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 0.9632
TP: 19 | TN: 9 | FP: 1 | FN: 0

model_1_seed_11.keras (evaluating on seed 11 test set: 29 samples)...
Acc: 0.8966 | Rec: 0.9474 | Prec: 0.9000 | F1: 0.9231 | AUC: 0.9842
TP: 18 | TN: 8 | FP: 2 | FN: 1

model_1_seed_12.keras (evaluating on seed 12 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_13.keras (evaluating on seed 13 test set: 29 samples)...
WARNING:tensorflow:5 out of the last 5 calls to <function
Model.make_predict_function.<locals>.predict_function at 0x000001D41E36C8B0>
triggered tf.function retracing. Tracing is expensive and the excessive number
of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2)
passing tensors with different shapes, (3) passing Python objects instead of
tensors. For (1), please define your @tf.function outside of the loop. For (2),
@tf.function has reduce_retracing=True option that can avoid unnecessary
retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for more details.
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_14.keras (evaluating on seed 14 test set: 29 samples)...
WARNING:tensorflow:6 out of the last 6 calls to <function
Model.make_predict_function.<locals>.predict_function at 0x000001D499970B80>
triggered tf.function retracing. Tracing is expensive and the excessive number
of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2)
passing tensors with different shapes, (3) passing Python objects instead of
tensors. For (1), please define your @tf.function outside of the loop. For (2),
@tf.function has reduce_retracing=True option that can avoid unnecessary
retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for more details.

Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_15.keras (evaluating on seed 15 test set: 29 samples)...
Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 1.0000
TP: 18 | TN: 10 | FP: 0 | FN: 1

model_1_seed_16.keras (evaluating on seed 16 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_17.keras (evaluating on seed 17 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_18.keras (evaluating on seed 18 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_19.keras (evaluating on seed 19 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_2.keras (evaluating on seed 2 test set: 29 samples)...
Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 0.9947
TP: 18 | TN: 10 | FP: 0 | FN: 1

model_1_seed_20.keras (evaluating on seed 20 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_21.keras (evaluating on seed 21 test set: 29 samples)...
Acc: 0.8966 | Rec: 0.8421 | Prec: 1.0000 | F1: 0.9143 | AUC: 0.9895
TP: 16 | TN: 10 | FP: 0 | FN: 3

model_1_seed_22.keras (evaluating on seed 22 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_23.keras (evaluating on seed 23 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_24.keras (evaluating on seed 24 test set: 29 samples)...
Acc: 0.8966 | Rec: 0.8947 | Prec: 0.9444 | F1: 0.9189 | AUC: 0.9895
TP: 17 | TN: 9 | FP: 1 | FN: 2

model_1_seed_25.keras (evaluating on seed 25 test set: 29 samples)...

Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_26.keras (evaluating on seed 26 test set: 29 samples)...
 Acc: 0.6207 | Rec: 0.4737 | Prec: 0.9000 | F1: 0.6207 | AUC: 0.8500
 TP: 9 | TN: 9 | FP: 1 | FN: 10

model_1_seed_27.keras (evaluating on seed 27 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_28.keras (evaluating on seed 28 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_29.keras (evaluating on seed 29 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_3.keras (evaluating on seed 3 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_30.keras (evaluating on seed 30 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_4.keras (evaluating on seed 4 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_5.keras (evaluating on seed 5 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_1_seed_6.keras (evaluating on seed 6 test set: 29 samples)...
 Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 1.0000
 TP: 19 | TN: 9 | FP: 1 | FN: 0

model_1_seed_7.keras (evaluating on seed 7 test set: 29 samples)...
 Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 0.9947
 TP: 18 | TN: 10 | FP: 0 | FN: 1

model_1_seed_8.keras (evaluating on seed 8 test set: 29 samples)...
 Acc: 0.8966 | Rec: 1.0000 | Prec: 0.8636 | F1: 0.9268 | AUC: 0.9500
 TP: 19 | TN: 7 | FP: 3 | FN: 0

model_1_seed_9.keras (evaluating on seed 9 test set: 29 samples)...

Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 1.0000
TP: 18 | TN: 10 | FP: 0 | FN: 1

model_2_seed_1.keras (evaluating on seed 1 test set: 29 samples)...
Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 1.0000
TP: 19 | TN: 9 | FP: 1 | FN: 0

model_2_seed_10.keras (evaluating on seed 10 test set: 29 samples)...
Acc: 0.9310 | Rec: 0.9474 | Prec: 0.9474 | F1: 0.9474 | AUC: 0.9447
TP: 18 | TN: 9 | FP: 1 | FN: 1

model_2_seed_11.keras (evaluating on seed 11 test set: 29 samples)...
Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 0.9474
TP: 19 | TN: 9 | FP: 1 | FN: 0

model_2_seed_12.keras (evaluating on seed 12 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_13.keras (evaluating on seed 13 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_14.keras (evaluating on seed 14 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_15.keras (evaluating on seed 15 test set: 29 samples)...
Acc: 0.9310 | Rec: 0.9474 | Prec: 0.9474 | F1: 0.9474 | AUC: 0.9474
TP: 18 | TN: 9 | FP: 1 | FN: 1

model_2_seed_16.keras (evaluating on seed 16 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_17.keras (evaluating on seed 17 test set: 29 samples)...
Acc: 0.7931 | Rec: 0.7368 | Prec: 0.9333 | F1: 0.8235 | AUC: 0.9105
TP: 14 | TN: 9 | FP: 1 | FN: 5

model_2_seed_18.keras (evaluating on seed 18 test set: 29 samples)...
Acc: 0.8276 | Rec: 1.0000 | Prec: 0.7917 | F1: 0.8837 | AUC: 0.9342
TP: 19 | TN: 5 | FP: 5 | FN: 0

model_2_seed_19.keras (evaluating on seed 19 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_2.keras (evaluating on seed 2 test set: 29 samples)...

Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 1.0000
TP: 19 | TN: 9 | FP: 1 | FN: 0

model_2_seed_20.keras (evaluating on seed 20 test set: 29 samples)..
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_21.keras (evaluating on seed 21 test set: 29 samples)..
Acc: 0.8621 | Rec: 1.0000 | Prec: 0.8261 | F1: 0.9048 | AUC: 0.8500
TP: 19 | TN: 6 | FP: 4 | FN: 0

model_2_seed_22.keras (evaluating on seed 22 test set: 29 samples)..
Acc: 0.8276 | Rec: 1.0000 | Prec: 0.7917 | F1: 0.8837 | AUC: 1.0000
TP: 19 | TN: 5 | FP: 5 | FN: 0

model_2_seed_23.keras (evaluating on seed 23 test set: 29 samples)..
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_24.keras (evaluating on seed 24 test set: 29 samples)..
Acc: 0.8966 | Rec: 0.8421 | Prec: 1.0000 | F1: 0.9143 | AUC: 1.0000
TP: 16 | TN: 10 | FP: 0 | FN: 3

model_2_seed_25.keras (evaluating on seed 25 test set: 29 samples)..
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_26.keras (evaluating on seed 26 test set: 29 samples)..
Acc: 0.8966 | Rec: 0.8947 | Prec: 0.9444 | F1: 0.9189 | AUC: 0.9447
TP: 17 | TN: 9 | FP: 1 | FN: 2

model_2_seed_27.keras (evaluating on seed 27 test set: 29 samples)..
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_28.keras (evaluating on seed 28 test set: 29 samples)..
Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 0.9474
TP: 19 | TN: 9 | FP: 1 | FN: 0

model_2_seed_29.keras (evaluating on seed 29 test set: 29 samples)..
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_3.keras (evaluating on seed 3 test set: 29 samples)..
Acc: 0.8621 | Rec: 1.0000 | Prec: 0.8261 | F1: 0.9048 | AUC: 1.0000
TP: 19 | TN: 6 | FP: 4 | FN: 0

model_2_seed_30.keras (evaluating on seed 30 test set: 29 samples)...

Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_4.keras (evaluating on seed 4 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_5.keras (evaluating on seed 5 test set: 29 samples)...
 Acc: 0.9310 | Rec: 1.0000 | Prec: 0.9048 | F1: 0.9500 | AUC: 1.0000
 TP: 19 | TN: 8 | FP: 2 | FN: 0

model_2_seed_6.keras (evaluating on seed 6 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_2_seed_7.keras (evaluating on seed 7 test set: 29 samples)...
 Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 0.9500
 TP: 19 | TN: 9 | FP: 1 | FN: 0

model_2_seed_8.keras (evaluating on seed 8 test set: 29 samples)...
 Acc: 0.8621 | Rec: 1.0000 | Prec: 0.8261 | F1: 0.9048 | AUC: 0.9789
 TP: 19 | TN: 6 | FP: 4 | FN: 0

model_2_seed_9.keras (evaluating on seed 9 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_1.keras (evaluating on seed 1 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_10.keras (evaluating on seed 10 test set: 29 samples)...
 Acc: 0.9310 | Rec: 0.8947 | Prec: 1.0000 | F1: 0.9444 | AUC: 0.9895
 TP: 17 | TN: 10 | FP: 0 | FN: 2

model_3_seed_11.keras (evaluating on seed 11 test set: 29 samples)...
 Acc: 0.8966 | Rec: 0.8947 | Prec: 0.9444 | F1: 0.9189 | AUC: 0.9895
 TP: 17 | TN: 9 | FP: 1 | FN: 2

model_3_seed_12.keras (evaluating on seed 12 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_13.keras (evaluating on seed 13 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_14.keras (evaluating on seed 14 test set: 29 samples)...

Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 1.0000
 TP: 19 | TN: 9 | FP: 1 | FN: 0

model_3_seed_15.keras (evaluating on seed 15 test set: 29 samples)...
 Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 0.9895
 TP: 19 | TN: 9 | FP: 1 | FN: 0

model_3_seed_16.keras (evaluating on seed 16 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_17.keras (evaluating on seed 17 test set: 29 samples)...
 Acc: 0.8621 | Rec: 0.7895 | Prec: 1.0000 | F1: 0.8824 | AUC: 0.9895
 TP: 15 | TN: 10 | FP: 0 | FN: 4

model_3_seed_18.keras (evaluating on seed 18 test set: 29 samples)...
 Acc: 0.9310 | Rec: 0.8947 | Prec: 1.0000 | F1: 0.9444 | AUC: 0.9895
 TP: 17 | TN: 10 | FP: 0 | FN: 2

model_3_seed_19.keras (evaluating on seed 19 test set: 29 samples)...
 Acc: 0.9655 | Rec: 1.0000 | Prec: 0.9500 | F1: 0.9744 | AUC: 1.0000
 TP: 19 | TN: 9 | FP: 1 | FN: 0

model_3_seed_2.keras (evaluating on seed 2 test set: 29 samples)...
 Acc: 0.9310 | Rec: 0.9474 | Prec: 0.9474 | F1: 0.9474 | AUC: 0.9289
 TP: 18 | TN: 9 | FP: 1 | FN: 1

model_3_seed_20.keras (evaluating on seed 20 test set: 29 samples)...
 Acc: 0.9310 | Rec: 0.9474 | Prec: 0.9474 | F1: 0.9474 | AUC: 0.9947
 TP: 18 | TN: 9 | FP: 1 | FN: 1

model_3_seed_21.keras (evaluating on seed 21 test set: 29 samples)...
 Acc: 0.8276 | Rec: 0.8421 | Prec: 0.8889 | F1: 0.8649 | AUC: 0.9026
 TP: 16 | TN: 8 | FP: 2 | FN: 3

model_3_seed_22.keras (evaluating on seed 22 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_23.keras (evaluating on seed 23 test set: 29 samples)...
 Acc: 0.6552 | Rec: 0.4737 | Prec: 1.0000 | F1: 0.6429 | AUC: 0.9895
 TP: 9 | TN: 10 | FP: 0 | FN: 10

model_3_seed_24.keras (evaluating on seed 24 test set: 29 samples)...
 Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
 TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_25.keras (evaluating on seed 25 test set: 29 samples)...

Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_26.keras (evaluating on seed 26 test set: 29 samples)...
Acc: 0.7241 | Rec: 0.6316 | Prec: 0.9231 | F1: 0.7500 | AUC: 0.8684
TP: 12 | TN: 9 | FP: 1 | FN: 7

model_3_seed_27.keras (evaluating on seed 27 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_28.keras (evaluating on seed 28 test set: 29 samples)...
Acc: 0.9310 | Rec: 1.0000 | Prec: 0.9048 | F1: 0.9500 | AUC: 0.9500
TP: 19 | TN: 8 | FP: 2 | FN: 0

model_3_seed_29.keras (evaluating on seed 29 test set: 29 samples)...
Acc: 0.6207 | Rec: 0.8947 | Prec: 0.6538 | F1: 0.7556 | AUC: 0.8421
TP: 17 | TN: 1 | FP: 9 | FN: 2

model_3_seed_3.keras (evaluating on seed 3 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_30.keras (evaluating on seed 30 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_4.keras (evaluating on seed 4 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_5.keras (evaluating on seed 5 test set: 29 samples)...
Acc: 0.9655 | Rec: 0.9474 | Prec: 1.0000 | F1: 0.9730 | AUC: 1.0000
TP: 18 | TN: 10 | FP: 0 | FN: 1

model_3_seed_6.keras (evaluating on seed 6 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_7.keras (evaluating on seed 7 test set: 29 samples)...
Acc: 1.0000 | Rec: 1.0000 | Prec: 1.0000 | F1: 1.0000 | AUC: 1.0000
TP: 19 | TN: 10 | FP: 0 | FN: 0

model_3_seed_8.keras (evaluating on seed 8 test set: 29 samples)...
Acc: 0.9310 | Rec: 1.0000 | Prec: 0.9048 | F1: 0.9500 | AUC: 0.9316
TP: 19 | TN: 8 | FP: 2 | FN: 0

model_3_seed_9.keras (evaluating on seed 9 test set: 29 samples)...

Acc: 0.9310 | Rec: 0.8947 | Prec: 1.0000 | F1: 0.9444 | AUC: 1.0000
TP: 17 | TN: 10 | FP: 0 | FN: 2

=====

=====

CALCULATING MEAN METRICS ACROSS SEEDS

=====

=====

=====

Model_1 - Statistics across 30 seeds

=====

Accuracy: 0.9655 ± 0.0729
Recall: 0.9632 ± 0.0981
Precision: 0.9836 ± 0.0359
F1-Score: 0.9706 ± 0.0703
AUC-ROC: 0.9904 ± 0.0284
Specificity: 0.9700 ± 0.0690
True Positives: 18.3000 ± 1.8646
True Negatives: 9.7000 ± 0.6904
False Positives: 0.3000 ± 0.6904
False Negatives: 0.7000 ± 1.8646

Representative Seed: 1 (Recall closest to mean)

=====

Model_2 - Statistics across 30 seeds

=====

Accuracy: 0.9483 ± 0.0634
Recall: 0.9789 ± 0.0570
Precision: 0.9496 ± 0.0673
F1-Score: 0.9618 ± 0.0472
AUC-ROC: 0.9785 ± 0.0357
Specificity: 0.8900 ± 0.1578
True Positives: 18.6000 ± 1.0832
True Negatives: 8.9000 ± 1.5780
False Positives: 1.1000 ± 1.5780
False Negatives: 0.4000 ± 1.0832

Representative Seed: 1 (Recall closest to mean)

=====

Model_3 - Statistics across 30 seeds

=====

Accuracy: 0.9322 ± 0.0997
Recall: 0.9351 ± 0.1182
Precision: 0.9655 ± 0.0675
F1-Score: 0.9446 ± 0.0854

AUC-ROC: 0.9785 $\pm$ 0.0411
 Specificity: 0.9267 $\pm$ 0.1672
 True Positives: 17.7667 $\pm$ 2.2462
 True Negatives: 9.2667 $\pm$ 1.6720
 False Positives: 0.7333 $\pm$ 1.6720
 False Negatives: 1.2333 $\pm$ 2.2462

Representative Seed: 2 (Recall closest to mean)

```
=====
=====
EVALUATION COMPLETE
=====
=====
```

```
evaluation_results populated with 3 models
  Model_1: Acc=0.9655 $\pm$ 0.0729, Rep.Seed=1
  Model_2: Acc=0.9483 $\pm$ 0.0634, Rep.Seed=1
  Model_3: Acc=0.9322 $\pm$ 0.0997, Rep.Seed=2
```

1.4 5.5 Ensemble Model - Mean Metrics Across Seeds

1.4.1 Ensemble Strategy

The ensemble was computed **per-seed** during training (in the multi-seed loop), using **Soft Voting** (average probabilities) across Models 1-3.

- **Mean Metrics:** We report the mean of ensemble metrics computed across all seeds
- **Representative Ensemble:** For visualizations requiring actual predictions, we use predictions from representative seeds
- **Goal:** Reduce false negatives by leveraging diverse model predictions while maintaining statistical rigor

```
[4]: # =====
# MODEL 4: ENSEMBLE - MEAN METRICS ACROSS SEEDS
# =====
# The Ensemble was already computed per-seed in the training loop and stored
# in multi_run_results. Here we calculate mean ensemble metrics and also
# compute ensemble predictions for ALL seeds to enable multi-seed
# visualizations.

print("\n" + "="*70)
print("MODEL 4: ENSEMBLE - MEAN METRICS ACROSS SEEDS")
print("="*70)

# =====
# COMPUTE ENSEMBLE FOR ALL SEEDS (for multi-seed visualizations)
# =====
print("\n" + "-"*70)
```



```

print("COMPUTING ENSEMBLE PREDICTIONS FOR ALL SEEDS")
print("-"*70)

# Add Ensemble to seed_metrics so it can be used in threshold sensitivity,
# calibration, etc.
seed_metrics['Ensemble'] = []

for seed_idx in range(1, len(EXPERIMENT_SEEDS) + 1):
    # Load all 3 models for this seed and compute ensemble
    seed_probas = []
    X_test_seed, y_test_seed = test_sets[seed_idx]
    X_test_seed_scaled = X_test_seed / 255.0

    for model_num in [1, 2, 3]:
        model_path = f'models/model_{model_num}_seed_{seed_idx}.keras'
        try:
            loaded_model = models.load_model(model_path)
            proba = loaded_model.predict(X_test_seed_scaled, verbose=0).
            #flatten()
            seed_probas.append(proba)
            K.clear_session()
            gc.collect()
        except:
            pass

    if len(seed_probas) == 3:
        # Compute ensemble via soft voting
        ens_proba = np.mean(seed_probas, axis=0)
        ens_pred = (ens_proba > 0.5).astype(int)

        # Calculate metrics
        e_acc = accuracy_score(y_test_seed, ens_pred)
        e_rec = recall_score(y_test_seed, ens_pred)
        e_prec = precision_score(y_test_seed, ens_pred)
        e_f1 = f1_score(y_test_seed, ens_pred)
        e_auc = roc_auc_score(y_test_seed, ens_proba)
        e_cm = confusion_matrix(y_test_seed, ens_pred)
        e_tn, e_fp, e_fn, e_tp = e_cm.ravel()
        e_spec = e_tn / (e_tn + e_fp) if (e_tn + e_fp) > 0 else 0

        seed_metrics['Ensemble'].append({
            'seed_idx': seed_idx,
            'Accuracy': e_acc,
            'Recall': e_rec,
            'Precision': e_prec,
            'F1-Score': e_f1,
            'AUC-ROC': e_auc,

```

```

        'Specificity': e_spec,
        'True Positives': e_tp,
        'True Negatives': e_tn,
        'False Positives': e_fp,
        'False Negatives': e_fn,
        'Predictions': ens_pred,
        'Probabilities': ens_proba,
        'Confusion Matrix': e_cm,
        'y_test': y_test_seed
    })
    print(f"    Seed {seed_idx}: Ensemble Acc={e_acc:.4f}, Rec={e_rec:.4f},  

    ↪F1={e_f1:.4f}")

print(f"\n Computed ensemble predictions for {len(seed_metrics['Ensemble'])}  

    ↪seeds")
print("    (Ensemble now available in seed_metrics for multi-seed,  

    ↪visualizations)")

# =====
# COMPUTE MEAN ENSEMBLE METRICS FROM seed_metrics['Ensemble']
# =====
print("\n" + "-"*70)
print("ENSEMBLE METRICS FROM MULTI-SEED EXPERIMENT")
print("-"*70)

# First, try to use multi_run_results if available (from training run)
ensemble_seed_results = [r for r in multi_run_results if r['Model'] ==  

    ↪'Ensemble'] if 'multi_run_results' in dir() else []

# If multi_run_results is empty, compute mean from seed_metrics['Ensemble']  

    ↪(always available)
if not ensemble_seed_results and seed_metrics['Ensemble']:
    print(f"\nComputing mean metrics from seed_metrics['Ensemble']  

    ↪({len(seed_metrics['Ensemble'])} seeds):")

    # Calculate mean and std for ensemble from seed_metrics
    metrics_to_avg = ['Accuracy', 'Recall', 'Precision', 'F1-Score', 'AUC-ROC',  

    ↪'Specificity',
                        'True Positives', 'True Negatives', 'False Positives',  

    ↪'False Negatives']

    ensemble_mean = {}
    ensemble_std = {}
    for metric in metrics_to_avg:
        values = [s[metric] for s in seed_metrics['Ensemble']]
        ensemble_mean[metric] = np.mean(values)

```

```

        ensemble_std[metric] = np.std(values)

    # Add aliases for compatibility
    ensemble_mean['TP'] = ensemble_mean['True Positives']
    ensemble_mean['TN'] = ensemble_mean['True Negatives']
    ensemble_mean['FP'] = ensemble_mean['False Positives']
    ensemble_mean['FN'] = ensemble_mean['False Negatives']

    print(f"\n    Mean Ensemble Metrics (from loaded models):")
    print(f"    Accuracy:    {ensemble_mean['Accuracy']:.4f} ±␣
↪{ensemble_std['Accuracy']:.4f}")
    print(f"    Recall:      {ensemble_mean['Recall']:.4f} ±␣
↪{ensemble_std['Recall']:.4f}")
    print(f"    Precision:   {ensemble_mean['Precision']:.4f} ±␣
↪{ensemble_std['Precision']:.4f}")
    print(f"    F1-Score:    {ensemble_mean['F1-Score']:.4f} ±␣
↪{ensemble_std['F1-Score']:.4f}")
    print(f"    AUC-ROC:     {ensemble_mean['AUC-ROC']:.4f} ±␣
↪{ensemble_std['AUC-ROC']:.4f}")
    print(f"    Specificity: {ensemble_mean['Specificity']:.4f} ±␣
↪{ensemble_std['Specificity']:.4f}")

    # Set flag so downstream code knows we have valid ensemble metrics
    ensemble_seed_results = seed_metrics['Ensemble']

elif ensemble_seed_results:
    print(f"\nEnsemble computed for {len(ensemble_seed_results)} seeds:")
    for r in ensemble_seed_results:
        auc_str = f", AUC={r.get('AUC-ROC', 'N/A'):.4f}" if 'AUC-ROC' in r else␣
↪""
        print(f"    Seed {r['Seed']}: Acc={r['Accuracy']:.4f}, Rec={r['Recall']:.
↪4f}, Prec={r['Precision']:.4f}, F1={r['F1']:.4f}{auc_str}")

    # Calculate mean and std for ensemble (including new metrics if available)
    ensemble_mean = {
        'Accuracy': np.mean([r['Accuracy'] for r in ensemble_seed_results]),
        'Recall': np.mean([r['Recall'] for r in ensemble_seed_results]),
        'Precision': np.mean([r['Precision'] for r in ensemble_seed_results]),
        'F1-Score': np.mean([r['F1'] for r in ensemble_seed_results])
    }
    ensemble_std = {
        'Accuracy': np.std([r['Accuracy'] for r in ensemble_seed_results]),
        'Recall': np.std([r['Recall'] for r in ensemble_seed_results]),
        'Precision': np.std([r['Precision'] for r in ensemble_seed_results]),
        'F1-Score': np.std([r['F1'] for r in ensemble_seed_results])
    }

```

```

# Check if AUC-ROC and Specificity were tracked per-seed
if 'AUC-ROC' in ensemble_seed_results[0]:
    ensemble_mean['AUC-ROC'] = np.mean([r['AUC-ROC'] for r in
↳ensemble_seed_results])
    ensemble_std['AUC-ROC'] = np.std([r['AUC-ROC'] for r in
↳ensemble_seed_results])
    ensemble_mean['Specificity'] = np.mean([r['Specificity'] for r in
↳ensemble_seed_results])
    ensemble_std['Specificity'] = np.std([r['Specificity'] for r in
↳ensemble_seed_results])

# Mean confusion matrix values
ensemble_mean['TP'] = np.mean([r['TP'] for r in ensemble_seed_results])
ensemble_mean['TN'] = np.mean([r['TN'] for r in ensemble_seed_results])
ensemble_mean['FP'] = np.mean([r['FP'] for r in ensemble_seed_results])
ensemble_mean['FN'] = np.mean([r['FN'] for r in ensemble_seed_results])

print(f"\n Mean Ensemble Metrics:")
print(f" Accuracy: {ensemble_mean['Accuracy']:.4f} ±
↳{ensemble_std['Accuracy']:.4f}")
print(f" Recall: {ensemble_mean['Recall']:.4f} ±
↳{ensemble_std['Recall']:.4f}")
print(f" Precision: {ensemble_mean['Precision']:.4f} ±
↳{ensemble_std['Precision']:.4f}")
print(f" F1-Score: {ensemble_mean['F1-Score']:.4f} ±
↳{ensemble_std['F1-Score']:.4f}")
if 'AUC-ROC' in ensemble_mean:
    print(f" AUC-ROC: {ensemble_mean['AUC-ROC']:.4f} ±
↳{ensemble_std['AUC-ROC']:.4f}")
    print(f" Specificity: {ensemble_mean['Specificity']:.4f} ±
↳{ensemble_std['Specificity']:.4f}")

# =====
# APPROACH 2: Compute ensemble from representative seed for visualizations
# =====

print("\n" + "-"*70)
print("ENSEMBLE FROM REPRESENTATIVE SEED (FOR VISUALIZATIONS)")
print("-"*70)

# Get representative seeds from each model
rep_seeds = [next(r for r in evaluation_results if r['Model'] ==
↳m) ['Representative_Seed']
               for m in ['Model_1', 'Model_2', 'Model_3']]

# Find the most common representative seed (or just use the first one)
from collections import Counter

```

```

most_common_seed = Counter(rep_seeds).most_common(1)[0][0]
print(f"\nRepresentative seeds from models: {rep_seeds}")
print(f"Using seed {most_common_seed} for ensemble visualization")

# Get test set for this representative seed
y_test_rep = test_sets[most_common_seed][1]

# Load models for this specific seed and compute ensemble
all_probabilities = []
print(f"\nLoading models from seed {most_common_seed}:")
for model_num in [1, 2, 3]:
    model_path = f'models/model_{model_num}_seed_{most_common_seed}.keras'
    print(f"    Loading {model_path}...")
    loaded_model = models.load_model(model_path)
    X_test_rep = test_sets[most_common_seed][0] / 255.0
    proba = loaded_model.predict(X_test_rep, verbose=0).flatten()
    all_probabilities.append(proba)
    K.clear_session()
    gc.collect()

all_probabilities = np.array(all_probabilities)

# Soft Voting: Average probabilities
ensemble_proba = np.mean(all_probabilities, axis=0)
ensemble_pred = (ensemble_proba > 0.5).astype(int)

# Calculate actual metrics for this representative ensemble using CORRECT test_
↳ labels
accuracy_ensemble = accuracy_score(y_test_rep, ensemble_pred)
precision_ensemble = precision_score(y_test_rep, ensemble_pred)
recall_ensemble = recall_score(y_test_rep, ensemble_pred)
f1_ensemble = f1_score(y_test_rep, ensemble_pred)
auc_ensemble = roc_auc_score(y_test_rep, ensemble_proba)

# Confusion matrix
cm_ensemble = confusion_matrix(y_test_rep, ensemble_pred)
tn, fp, fn, tp = cm_ensemble.ravel()
specificity_ensemble = tn / (tn + fp) if (tn + fp) > 0 else 0

print(f"\nRepresentative Ensemble Performance (Seed {most_common_seed},
↳ {len(y_test_rep)} test samples):")
print(f"    Accuracy:    {accuracy_ensemble:.4f}")
print(f"    Recall:       {recall_ensemble:.4f}")
print(f"    Precision:    {precision_ensemble:.4f}")
print(f"    F1-Score:     {f1_ensemble:.4f}")
print(f"    AUC-ROC:      {auc_ensemble:.4f}")
print(f"    Specificity:  {specificity_ensemble:.4f}")

```

```

# =====
# ADD ENSEMBLE TO EVALUATION RESULTS
# =====
print("\n" + "="*70)
print("ADDING ENSEMBLE TO EVALUATION RESULTS")
print("="*70)

# Use MEAN METRICS from multi_run_results if available, otherwise use
↳ representative
if ensemble_seed_results:
    # Check if full metrics were tracked (AUC-ROC, Specificity, confusion
    ↳ matrix)
    has_full_metrics = 'AUC-ROC' in ensemble_mean

    evaluation_results.append({
        'Model': 'Model_4_Ensemble',
        # Use mean metrics from multi-seed experiment
        'Accuracy': ensemble_mean['Accuracy'],
        'Precision': ensemble_mean['Precision'],
        'Recall': ensemble_mean['Recall'],
        'F1-Score': ensemble_mean['F1-Score'],
        # Use mean AUC/Specificity if tracked, otherwise from representative
        'AUC-ROC': ensemble_mean.get('AUC-ROC', auc_ensemble),
        'Specificity': ensemble_mean.get('Specificity', specificity_ensemble),
        # Confusion matrix values: use mean if tracked, otherwise from
        ↳ representative
        'True Positives': int(round(ensemble_mean.get('TP', tp))),
        'True Negatives': int(round(ensemble_mean.get('TN', tn))),
        'False Positives': int(round(ensemble_mean.get('FP', fp))),
        'False Negatives': int(round(ensemble_mean.get('FN', fn))),
        'Predictions': ensemble_pred,
        'Probabilities': ensemble_proba,
        'Confusion Matrix': cm_ensemble,
        'y_test': y_test_rep,
        # Store std for error bars
        'Accuracy_std': ensemble_std['Accuracy'],
        'Recall_std': ensemble_std['Recall'],
        'Precision_std': ensemble_std['Precision'],
        'F1-Score_std': ensemble_std['F1-Score'],
        'AUC-ROC_std': ensemble_std.get('AUC-ROC', 0.0),
        'Specificity_std': ensemble_std.get('Specificity', 0.0),
        'Representative_Seed': most_common_seed
    })

    print(f"\n Added Ensemble with MEAN metrics from
    ↳ {len(ensemble_seed_results)} seeds")

```

```

    if has_full_metrics:
        print(f"    (Including mean AUC-ROC, Specificity, and confusion matrix,
↳values)")
    else:
        # Fallback: use representative ensemble metrics
        evaluation_results.append({
            'Model': 'Model_4_Ensemble',
            'Accuracy': accuracy_ensemble,
            'Precision': precision_ensemble,
            'Recall': recall_ensemble,
            'F1-Score': f1_ensemble,
            'AUC-ROC': auc_ensemble,
            'Specificity': specificity_ensemble,
            'True Positives': tp,
            'True Negatives': tn,
            'False Positives': fp,
            'False Negatives': fn,
            'Predictions': ensemble_pred,
            'Probabilities': ensemble_proba,
            'Confusion Matrix': cm_ensemble,
            'y_test': y_test_rep,
            'Accuracy_std': 0.0,
            'Recall_std': 0.0,
            'Precision_std': 0.0,
            'F1-Score_std': 0.0,
            'AUC-ROC_std': 0.0,
            'Specificity_std': 0.0,
            'Representative_Seed': most_common_seed
        })
        print(f"\n Added Ensemble from representative seed {most_common_seed}")

# =====
# FINAL SUMMARY
# =====
print("\n" + "="*70)
print("UPDATED EVALUATION SUMMARY - ALL 4 MODELS")
print("="*70)

eval_df = pd.DataFrame(evaluation_results)
eval_summary = eval_df[['Model', 'Accuracy', 'Precision', 'Recall', 'F1-Score',
↳'AUC-ROC', 'Specificity']].copy()
print(eval_summary.to_string(index=False))

# =====
# ENSEMBLE IMPROVEMENT ANALYSIS
# =====
print("\n" + "="*70)

```

```

print("ENSEMBLE IMPROVEMENT ANALYSIS")
print("="*70)

best_individual_recall = max([r['Recall'] for r in evaluation_results if
    ↳ 'Model_' in r['Model'] and 'Ensemble' not in r['Model']])
ensemble_recall = next(r['Recall'] for r in evaluation_results if r['Model'] ==
    ↳ 'Model_4_Ensemble')
recall_improvement = ensemble_recall - best_individual_recall

print(f"\nBest Individual Model Recall (mean): {best_individual_recall:.4f}")
print(f"Ensemble Model Recall (mean):          {ensemble_recall:.4f}")
print(f"Improvement:                            {recall_improvement:+.4f}
    ↳ ({recall_improvement/best_individual_recall*100:+.2f}%)")

if recall_improvement > 0:
    print(f"\n Ensemble IMPROVED infected fish detection!")
elif recall_improvement == 0:
    print(f"\n Ensemble MATCHED best individual model performance")
else:
    print(f"\n Ensemble did not improve over best individual model")

print(f"\n Mean False Negatives: {ensemble_mean.get('FN', fn):.1f} missed
    ↳ infections")

```

MODEL 4: ENSEMBLE - MEAN METRICS ACROSS SEEDS

COMPUTING ENSEMBLE PREDICTIONS FOR ALL SEEDS

```

Seed 1: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 2: Ensemble Acc=0.9655, Rec=1.0000, F1=0.9744
Seed 3: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 4: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 5: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 6: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 7: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 8: Ensemble Acc=0.9310, Rec=1.0000, F1=0.9500
Seed 9: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 10: Ensemble Acc=0.9310, Rec=0.9474, F1=0.9474
Seed 11: Ensemble Acc=0.9310, Rec=0.9474, F1=0.9474
Seed 12: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 13: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 14: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 15: Ensemble Acc=0.9655, Rec=1.0000, F1=0.9744

```


Seed 16: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 17: Ensemble Acc=0.9310, Rec=0.8947, F1=0.9444
Seed 18: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 19: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 20: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 21: Ensemble Acc=0.8966, Rec=0.9474, F1=0.9231
Seed 22: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 23: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 24: Ensemble Acc=0.9310, Rec=0.8947, F1=0.9444
Seed 25: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 26: Ensemble Acc=0.7586, Rec=0.6842, F1=0.7879
Seed 27: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 28: Ensemble Acc=0.9655, Rec=1.0000, F1=0.9744
Seed 29: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000
Seed 30: Ensemble Acc=1.0000, Rec=1.0000, F1=1.0000

Computed ensemble predictions for 30 seeds

(Ensemble now available in seed_metrics for multi-seed visualizations)

ENSEMBLE METRICS FROM MULTI-SEED EXPERIMENT

Computing mean metrics from seed_metrics['Ensemble'] (30 seeds):

Mean Ensemble Metrics (from loaded models):

Accuracy: 0.9736 ± 0.0501
Recall: 0.9772 ± 0.0618
Precision: 0.9826 ± 0.0306
F1-Score: 0.9789 ± 0.0423
AUC-ROC: 0.9941 ± 0.0210
Specificity: 0.9667 ± 0.0596

ENSEMBLE FROM REPRESENTATIVE SEED (FOR VISUALIZATIONS)

Representative seeds from models: [1, 1, 2]

Using seed 1 for ensemble visualization

Loading models from seed 1:

Loading models/model_1_seed_1.keras...
Loading models/model_2_seed_1.keras...
Loading models/model_3_seed_1.keras...

Representative Ensemble Performance (Seed 1, 29 test samples):

Accuracy: 1.0000
Recall: 1.0000

Precision: 1.0000
F1-Score: 1.0000
AUC-ROC: 1.0000
Specificity: 1.0000

=====

ADDING ENSEMBLE TO EVALUATION RESULTS

=====

Added Ensemble with MEAN metrics from 30 seeds
(Including mean AUC-ROC, Specificity, and confusion matrix values)

=====

UPDATED EVALUATION SUMMARY - ALL 4 MODELS

=====

	Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Specificity
	Model_1	0.965517	0.983603	0.963158	0.970579	0.990351	0.970000
	Model_2	0.948276	0.949629	0.978947	0.961833	0.978509	0.890000
	Model_3	0.932184	0.965484	0.935088	0.944622	0.978509	0.926667
	Model_4_Ensemble	0.973563	0.982602	0.977193	0.978922	0.994123	0.966667

=====

ENSEMBLE IMPROVEMENT ANALYSIS

=====

Best Individual Model Recall (mean): 0.9789
Ensemble Model Recall (mean): 0.9772
Improvement: -0.0018 (-0.18%)

Ensemble did not improve over best individual model

Mean False Negatives: 0.4 missed infections

```
[ ]: # Visualize confusion matrices for all models (using MEAN values from seeds)

n_models = len(evaluation_results)
fig, axes = plt.subplots(2, 2, figsize=(10, 9))
axes = np.array(axes).reshape(-1)

for idx, result in enumerate(evaluation_results):
    ax = axes[idx]

    # Build confusion matrix from MEAN TP/TN/FP/FN values
    tn = result['True Negatives']
    fp = result['False Positives']
    fn = result['False Negatives']
    tp = result['True Positives']
```

```

cm = np.array([[tn, fp], [fn, tp]])

# Create heatmap
sns.heatmap(cm, annot=True, fmt='.1f', cmap='Blues', ax=ax,
            cbar=True, square=True)

ax.set_xlabel('Predicted')
ax.set_ylabel('True')
ax.set_title(result['Model'])
ax.set_xticklabels(['Fresh', 'Infected'])
ax.set_yticklabels(['Fresh', 'Infected'])

# Hide any unused subplots
for j in range(len(evaluation_results), len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.savefig('report/figures/confusion_matrices.png', dpi=300,
           bbox_inches='tight', facecolor='white')
plt.show()

# Performance comparison - single grouped bar chart
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
model_names = eval_summary['Model'].values
x = np.arange(len(model_names))
width = 0.2

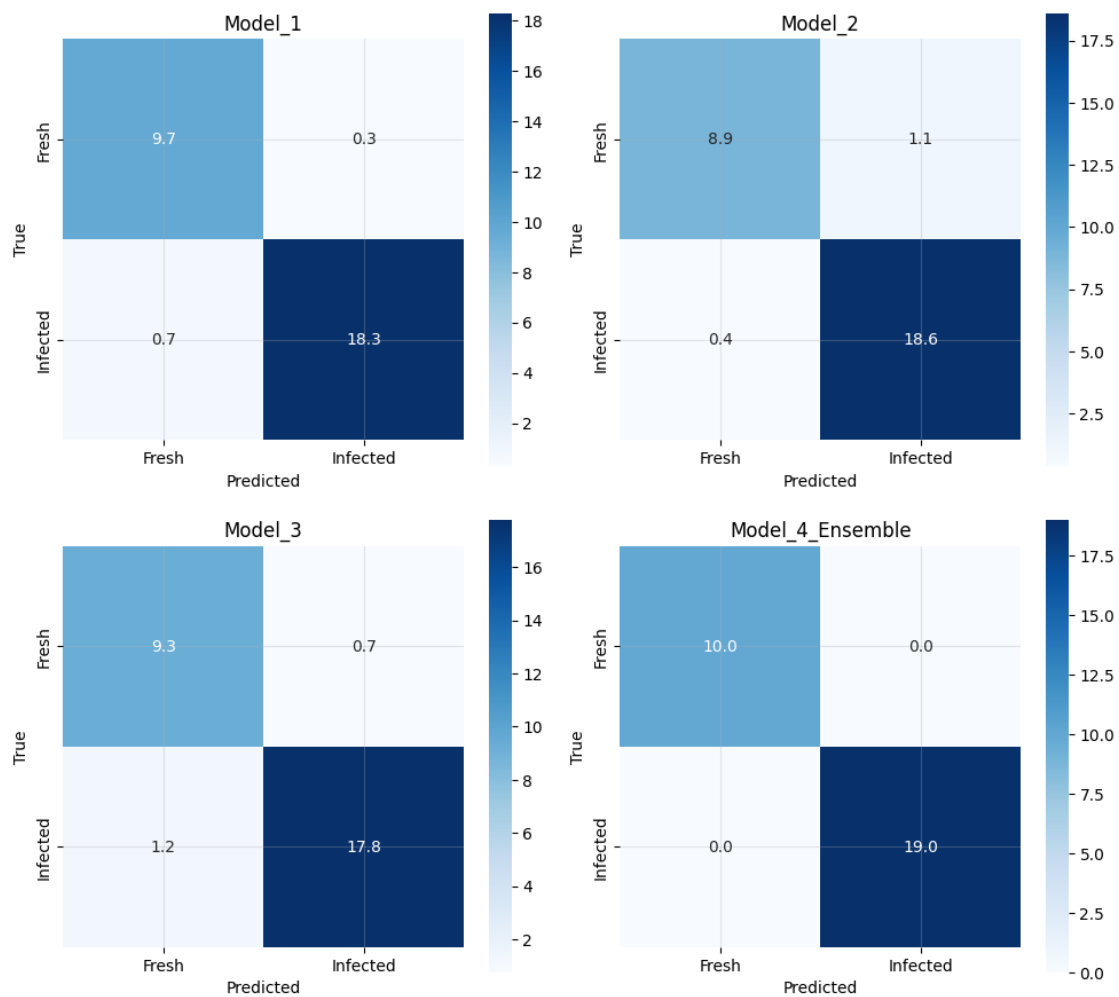
fig, ax = plt.subplots(figsize=(10, 6))

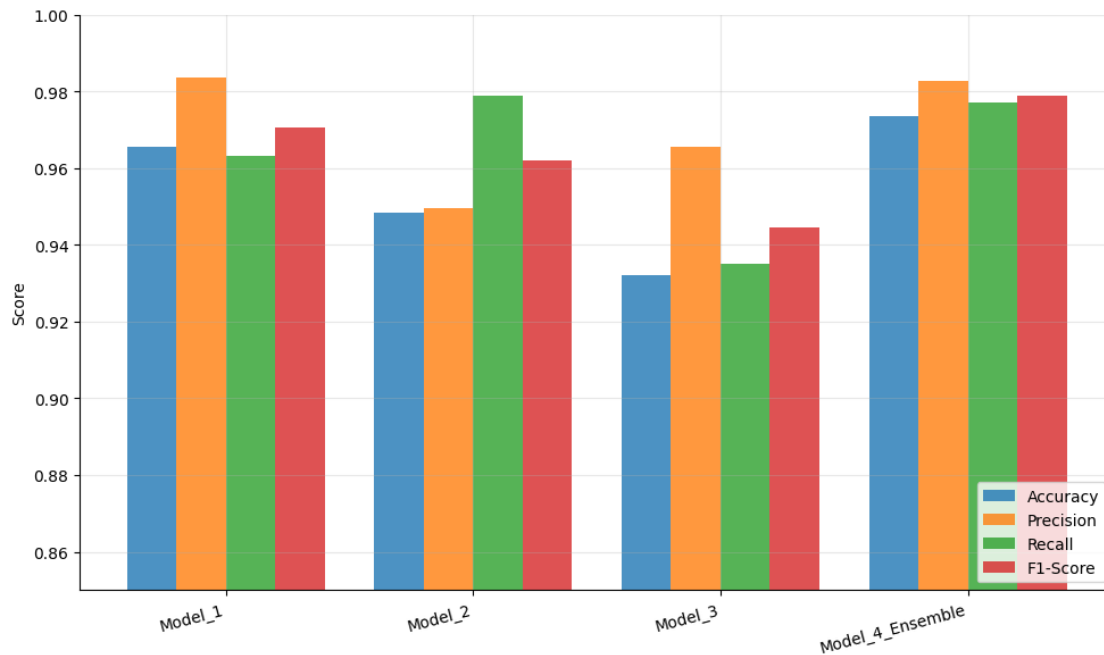
for i, metric in enumerate(metrics):
    values = eval_summary[metric].values
    ax.bar(x + i*width, values, width, label=metric, alpha=0.8)

ax.set_ylabel('Score')
ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(model_names, rotation=15, ha='right')
ax.legend(loc='lower right')
ax.set_ylim([0.85, 1.0])
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('report/figures/performance_comparison.png', dpi=300,
           bbox_inches='tight', facecolor='white')
plt.show()

```





```
[ ]: # =====
# ROC CURVES WITH ALL SEEDS - Shows variability across train-test splits
# =====
from sklearn.metrics import roc_curve

print("="*80)
print("ROC CURVES: ALL SEEDS OVERLAID WITH MEAN")
print("="*80)

# Single combined plot with all models
fig, ax = plt.subplots(figsize=(8, 7))

model_colors = {'Model_1': '#3498db', 'Model_2': '#2ecc71', 'Model_3': '#e74c3c'}

for model_name in ['Model_1', 'Model_2', 'Model_3']:
    model_data = seed_metrics[model_name]

    mean_fpr = np.linspace(0, 1, 100)
    all_tprs = []
    all_aucs = []

    for seed_data in model_data:
        y_true = seed_data['y_test']
        y_proba = seed_data['Probabilities']
```

```

fpr, tpr, _ = roc_curve(y_true, y_proba)
all_aucs.append(seed_data['AUC-ROC'])

tpr_interp = np.interp(mean_fpr, fpr, tpr)
tpr_interp[0] = 0.0
all_tprs.append(tpr_interp)

mean_tpr = np.mean(all_tprs, axis=0)
mean_tpr[-1] = 1.0
mean_auc = np.mean(all_aucs)
std_auc = np.std(all_aucs)

ax.plot(mean_fpr, mean_tpr, color=model_colors[model_name], linewidth=2.5,
        label=f'{model_name} (AUC={mean_auc:.3f}±{std_auc:.3f})')

ax.plot([0, 1], [0, 1], 'k--', linewidth=1.5, alpha=0.5)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
ax.set_xlim([0.0, 1.0])
ax.set_ylim([0.0, 1.05])

plt.tight_layout()
plt.savefig('report/figures/roc_curves_all_seeds.png', dpi=300,
           bbox_inches='tight', facecolor='white')
plt.show()

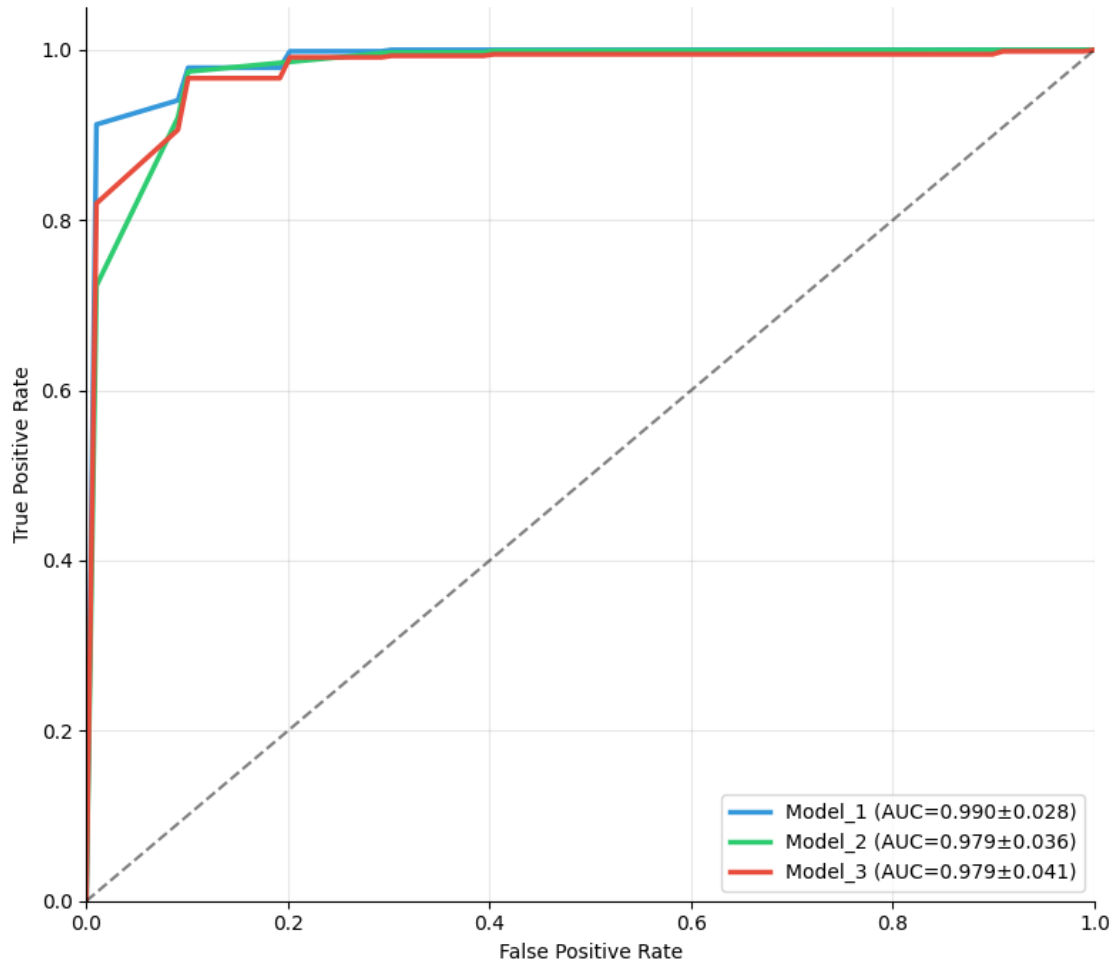
# Print summary
print("\nROC-AUC Summary (Mean ± Std across seeds):")
for model_name in ['Model_1', 'Model_2', 'Model_3']:
    auks = [s['AUC-ROC'] for s in seed_metrics[model_name]]
    print(f"  {model_name}: {np.mean(auks):.4f} ± {np.std(auks):.4f}")

```

```

=====
ROC CURVES: ALL SEEDS OVERLAID WITH MEAN
=====

```



ROC-AUC Summary (Mean ± Std across seeds):

Model_1: 0.9904 ± 0.0284

Model_2: 0.9785 ± 0.0357

Model_3: 0.9785 ± 0.0411

6. Six Sigma DMAIC Analysis

DMAIC Phase 4: IMPROVE

Step 13: Calculate Six Sigma Metrics and Process Capability

Six Sigma methodology uses statistical analysis to evaluate process quality:

- **Defects Per Million Opportunities (DPMO)**
- **Sigma Level:** Measures process capability (higher is better)
- **Process Capability Indices (Cp, Cpk)**

```

[7]: # Calculate Six Sigma metrics for each model
from scipy import stats

def calculate_sigma_level(accuracy):
    """
    Calculate Sigma level from accuracy
    Sigma Level is related to DPMO (Defects Per Million Opportunities)
    Includes standard 1.5 sigma shift for long-term process capability.
    """
    defect_rate = 1 - accuracy
    dpmo = defect_rate * 1_000_000

    # Calculate Z-score (sigma level)
    # Using standard normal distribution
    if defect_rate >= 0.5:
        sigma_level = 0
    elif defect_rate <= 0.00000001:
        sigma_level = 6.0
    else:
        # Z-score from defect rate (Long Term Z)
        z_score = stats.norm.ppf(1 - defect_rate)
        # Add 1.5 shift to get Sigma Level (Short Term Z)
        sigma_level = z_score + 1.5

    return sigma_level, dpmo

def calculate_process_capability(y_true, y_pred):
    """
    Calculate process capability indices Cp and Cpk
    These measure how well the process fits within specification limits
    """
    # Use accuracy as process metric
    accuracy = accuracy_score(y_true, y_pred)

    # For classification: target = 1.0 (perfect accuracy)
    # Specification limits: LSL = 0, USL = 1.0
    USL = 1.0
    LSL = 0.0
    target = 1.0

    # Standard deviation of prediction errors
    errors = (y_true != y_pred).astype(float)
    std_errors = np.std(errors)

    # Cap Cp and Cpk at reasonable maximum values for visualization
    # A very high Cpk (>3) indicates excellent process capability
    MAX_CPK = 3.0

```



```

if std_errors == 0 or std_errors < 1e-10:
    # Perfect predictions - cap at maximum
    Cp = Cpk = MAX_CPK
else:
    # Cp: Process capability (assumes process is centered)
    Cp = (USL - LSL) / (6 * std_errors)

    # Cpk: Process capability accounting for centering
    mean_process = accuracy
    Cpk = min((USL - mean_process) / (3 * std_errors),
              (mean_process - LSL) / (3 * std_errors))

    # Cap at reasonable maximum for visualization
    Cp = min(Cp, MAX_CPK)
    Cpk = min(Cpk, MAX_CPK)

return Cp, Cpk

# Calculate Six Sigma metrics for all models
# FOCUS ON INFECTED FISH DETECTION (Recall) as primary quality metric
six_sigma_results = []

print("\n" + "="*80)
print("  SIX SIGMA ANALYSIS - INFECTED FISH DETECTION FOCUS")
print("="*80)
print("Primary Metric: RECALL (Sensitivity for Infected Class)")
print("Rationale: False Negatives (missed infections) are catastrophic")
print("="*80 + "\n")

for result in evaluation_results:
    model_name = result['Model']
    accuracy = result['Accuracy']
    recall = result['Recall'] # Primary metric for infected detection
    y_pred = result['Predictions']
    y_test_result = result['y_test'] # Use correct test labels

    # Calculate Sigma metrics based on RECALL (infected fish detection rate)
    sigma_level_recall, dpmo_recall = calculate_sigma_level(recall)

    # Also calculate overall sigma for comparison
    sigma_level_overall, dpmo_overall = calculate_sigma_level(accuracy)

    # Calculate process capability using correct test labels
    Cp, Cpk = calculate_process_capability(y_test_result, y_pred)

```

```

    # Get confusion matrix values (use MEAN values, not representative seed's
    ↪CM)
    fn = result['False Negatives']
    fp = result['False Positives']
    tn = result['True Negatives']
    tp = result['True Positives']

    six_sigma_results.append({
        'Model': model_name,
        'Recall (Infected)': recall, # PRIMARY METRIC
        'Accuracy (Overall)': accuracy,
        'Infected DPMO': dpmo_recall, # Defects in infected detection
        'Overall DPMO': dpmo_overall,
        'Infected Sigma': sigma_level_recall, # PRIMARY SIGMA
        'Overall Sigma': sigma_level_overall,
        'Cp': Cp,
        'Cpk': Cpk,
        'False Negatives': fn, # CRITICAL DEFECT
        'False Positives': fp,
        'Total Infected': fn + tp,
        'Total Opportunities': len(y_test_result)
    })

# Create Six Sigma summary DataFrame
six_sigma_df = pd.DataFrame(six_sigma_results)

print("="*100)
print(" SIX SIGMA QUALITY METRICS - INFECTED FISH DETECTION FOCUS")
print("="*100)
print(six_sigma_df[['Model', 'Recall (Infected)', 'Infected DPMO', 'Infected_
    ↪Sigma',
                    'False Negatives', 'Accuracy (Overall)', 'Overall Sigma']]).
    ↪to_string(index=False))
print("\n" + "="*100)
print(" CRITICAL: 'Infected Sigma' is the primary quality metric")
print(" False Negatives (missed infections) cause disease outbreaks")
print("="*100)

# Interpretation guide
print("\nSIX SIGMA INTERPRETATION (for Infected Fish Detection):")
print("-" * 100)
print("Sigma Level → Infected Detection Rate → Business Impact:")
print(" 1 = 691,462 DPMO (31% detection) → UNACCEPTABLE - Most infections_
    ↪missed")
print(" 2 = 308,538 DPMO (69% detection) → POOR - High disease spread risk")
print(" 3 = 66,807 DPMO (93% detection) → ACCEPTABLE - Minimum for_
    ↪deployment")

```

```

print(" 4 = 6,210 DPMO (99.4% detection) → GOOD - Suitable for production")
print(" 5 = 233 DPMO (99.977% detection) → EXCELLENT - Industry leading")
print(" 6 = 3.4 DPMO (99.99966% detection) → WORLD CLASS - Near_
↳perfect")
print("\nProcess Capability (Cpk) interpretation:")
print(" Cpk < 1.0: Process not capable (poor)")
print(" Cpk = 1.0: Process minimally capable")
print(" Cpk = 1.33: Process capable (acceptable)")
print(" Cpk > 1.67: Process highly capable (good)")
print(" Cpk > 2.0: Process excellent")

# Save for CRISP-DM report
six_sigma_df.to_csv('six_sigma_analysis.csv', index=False)
print("\n Six Sigma results saved to 'six_sigma_analysis.csv'")

```

SIX SIGMA ANALYSIS - INFECTED FISH DETECTION FOCUS

Primary Metric: RECALL (Sensitivity for Infected Class)

Rationale: False Negatives (missed infections) are catastrophic

SIX SIGMA QUALITY METRICS - INFECTED FISH DETECTION FOCUS

	Model Recall (Infected)	Infected DPMO	Infected Sigma	False Negatives
Model_1	0.963158	36842.105263	3.288569	0.700000
Model_2	0.978947	21052.631579	3.532478	0.400000
Model_3	0.935088	64912.280702	3.014794	1.233333
Model_4_Ensemble	0.977193	22807.017544	3.498947	0.000000
	0.973563	3.435952		

CRITICAL: 'Infected Sigma' is the primary quality metric

False Negatives (missed infections) cause disease outbreaks

SIX SIGMA INTERPRETATION (for Infected Fish Detection):

Sigma Level → Infected Detection Rate → Business Impact:

1	= 691,462 DPMO (31% detection)	→ UNACCEPTABLE - Most infections missed
2	= 308,538 DPMO (69% detection)	→ POOR - High disease spread risk
3	= 66,807 DPMO (93% detection)	→ ACCEPTABLE - Minimum for deployment
4	= 6,210 DPMO (99.4% detection)	→ GOOD - Suitable for production
5	= 233 DPMO (99.977% detection)	→ EXCELLENT - Industry leading
6	= 3.4 DPMO (99.99966% detection)	→ WORLD CLASS - Near perfect

Process Capability (Cpk) interpretation:

Cpk < 1.0: Process not capable (poor)
Cpk = 1.0: Process minimally capable
Cpk = 1.33: Process capable (acceptable)
Cpk > 1.67: Process highly capable (good)
Cpk > 2.0: Process excellent

Six Sigma results saved to 'six_sigma_analysis.csv'

6.1. Advanced Model Analysis

Additional analyses for comprehensive CRISP-DM report:

- Learning curve analysis (diagnose over/underfitting)
- Error case analysis (which images are difficult?)
- Confidence distribution analysis
- Per-class performance breakdown

```
[13]: # =====  
# CROSS-SEED DIFFICULT CASES ANALYSIS  
# =====  
# Track which ORIGINAL IMAGES (by filename) are consistently misclassified  
# across different train-test splits. This reveals truly difficult samples.  
  
print("\n" + "="*80)  
print("CROSS-SEED DIFFICULT CASES ANALYSIS")  
print("="*80)  
print("Tracking which original images are misclassified across multiple seeds")  
print("(Images that appear in test set AND are misclassified)")  
print("="*80)  
  
# Build a mapping from original image index to filename  
# filenames was loaded with the dataset  
image_misclassification_tracker = {fname: {'times_in_test': 0,  
    ↪ 'times_misclassified': 0,  
    ↪ 'true_label': y[i], 'models_wrong':  
    ↪ [], 'orig_idx': i}  
    for i, fname in enumerate(filenamees)}
```

```

# For each seed, we need to track which original images were in the test set
# and whether they were misclassified
for model_name in ['Model_1', 'Model_2', 'Model_3']:
    for seed_data in seed_metrics[model_name]:
        seed_idx = seed_data['seed_idx']
        y_test_seed = seed_data['y_test']
        y_pred_seed = seed_data['Predictions']

        # Get the original indices for this seed's test set
        np.random.seed(EXPERIMENT_SEEDS[seed_idx - 1])
        _, test_indices = train_test_split(
            np.arange(len(X)), test_size=TEST_SIZE, stratify=y,
            random_state=EXPERIMENT_SEEDS[seed_idx - 1]
        )

        # Track misclassifications by original filename
        for i, orig_idx in enumerate(test_indices):
            fname = filenames[orig_idx]
            image_misclassification_tracker[fname]['times_in_test'] += 1

            if y_pred_seed[i] != y_test_seed[i]:
                image_misclassification_tracker[fname]['times_misclassified']_
↪ += 1
                image_misclassification_tracker[fname]['models_wrong'].
↪ append(f"{model_name}_s{seed_idx}")

# Find consistently difficult images
# An image is "difficult" if it's misclassified more than 30% of the times it_
↪ appears in test sets
difficult_images = []
for fname, stats in image_misclassification_tracker.items():
    if stats['times_in_test'] > 0:
        misclass_rate = stats['times_misclassified'] / stats['times_in_test']
        if stats['times_misclassified'] >= 2: # At least 2 misclassifications
            difficult_images.append({
                'filename': fname,
                'true_label': 'Fresh' if stats['true_label'] == 0 else_
↪ 'Infected',
                'true_label_int': stats['true_label'],
                'times_in_test': stats['times_in_test'],
                'times_misclassified': stats['times_misclassified'],
                'misclass_rate': misclass_rate,
                'models_wrong': stats['models_wrong'],
                'orig_idx': stats['orig_idx']
            })

```

```

# Sort by misclassification count
difficult_images.sort(key=lambda x: (-x['times_misclassified'],
↳ -x['misclass_rate']))

print(f"\nFound {len(difficult_images)} images misclassified 2+ times across
↳ all seed/model combinations:\n")

if difficult_images:
    print(f"{'Filename':<30} {'True Label':<12} {'In Test':<10}
↳ {'Misclassified':<15} {'Rate':<10}")
    print("-" * 80)
    for img in difficult_images[:15]: # Show top 15
        print(f"{img['filename']:<30} {img['true_label']:<12}
↳ {img['times_in_test']:<10} "
              f"{img['times_misclassified']:<15} {img['misclass_rate']:.1%}")

    # Breakdown by class
    difficult_fresh = [img for img in difficult_images if img['true_label'] ==
↳ 'Fresh']
    difficult_infected = [img for img in difficult_images if img['true_label']
↳ == 'Infected']

    print(f"\n" + "="*80)
    print("DIFFICULT IMAGES BY CLASS:")
    print("="*80)
    print(f" Difficult Fresh Fish: {len(difficult_fresh)} images (False
↳ Positives)")
    print(f" Difficult Infected Fish: {len(difficult_infected)} images (False
↳ Negatives - CRITICAL)")

    if difficult_infected:
        print(f"\n CRITICAL: These infected fish are consistently missed:")
        for img in difficult_infected[:5]:
            print(f" {img['filename']}: misclassified
↳ {img['times_misclassified']}/{img['times_in_test']} times
↳ ({img['misclass_rate']:.0%}")
    else:
        print("No images were consistently misclassified across seeds!")
        print("This suggests the models generalize well to different train-test
↳ splits.")

# Note: Grad-CAM explainability analysis was explored but yielded
↳ low-activation
# heatmaps across most samples, suggesting the models rely on distributed
↳ rather
# than localized features for classification. This is common with small datasets

```

```
# and shallow CNN architectures.
```

```
=====
CROSS-SEED DIFFICULT CASES ANALYSIS
=====
```

```
Tracking which original images are misclassified across multiple seeds
(Images that appear in test set AND are misclassified)
=====
```

Found 25 images misclassified 2+ times across all seed/model combinations:

Filename	True Label	In Test	Misclassified	Rate
a_salmon_fresh_10.png	Fresh	27	11	40.7%
salmon_dis_72.png	Infected	36	11	30.6%
fresh_31.png	Fresh	21	10	47.6%
fresh_34.png	Fresh	24	10	41.7%
salmon_dis_58.png	Infected	27	7	25.9%
a_salmon_fresh_04.png	Fresh	21	5	23.8%
salmon_dis_73.png	Infected	18	4	22.2%
salmon_dis_91.png	Infected	24	4	16.7%
salmon_dis_78.png	Infected	27	4	14.8%
a_salmon_fresh_09.png	Fresh	39	4	10.3%
salmon_dis_18.png	Infected	21	3	14.3%
a_salmon_fresh_05.png	Fresh	24	3	12.5%
a_salmon_fresh_24.png	Fresh	24	3	12.5%
salmon_dis_22.png	Infected	24	3	12.5%
salmon_dis_71.png	Infected	24	3	12.5%

```
=====
DIFFICULT IMAGES BY CLASS:
=====
```

Difficult Fresh Fish: 10 images (False Positives)

Difficult Infected Fish: 15 images (False Negatives - CRITICAL)

CRITICAL: These infected fish are consistently missed:

salmon_dis_72.png: misclassified 11/36 times (31%)

salmon_dis_58.png: misclassified 7/27 times (26%)

salmon_dis_73.png: misclassified 4/18 times (22%)

salmon_dis_91.png: misclassified 4/24 times (17%)

salmon_dis_78.png: misclassified 4/27 times (15%)

```
[15]: # Visualize Six Sigma metrics - simplified 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6']
```

```

# Plot 1: Sigma Level comparison
ax = axes[0, 0]
x = np.arange(len(six_sigma_df))
width = 0.35
ax.bar(x - width/2, six_sigma_df['Infected Sigma'], width, label='Infected',
       color='#e74c3c', alpha=0.9)
ax.bar(x + width/2, six_sigma_df['Overall Sigma'], width, label='Overall',
       color='#3498db', alpha=0.6)
ax.set_ylabel('Sigma Level ( )')
ax.set_xticks(x)
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
ax.axhline(y=3, color='orange', linestyle='--', linewidth=1.5, alpha=0.7)
ax.axhline(y=4, color='green', linestyle='--', linewidth=1.5, alpha=0.7)
ax.legend(loc='upper left', fontsize=9)
ax.grid(axis='y', alpha=0.3)

# Plot 2: False Negatives
ax = axes[0, 1]
bars = ax.bar(six_sigma_df['Model'], six_sigma_df['False Negatives'],
              color='#e74c3c', alpha=0.8)
ax.set_ylabel('False Negatives')
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
ax.grid(axis='y', alpha=0.3)
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height, f'{int(height)}',
           ha='center', va='bottom', fontsize=9)

# Plot 3: Process Capability (Cpk)
ax = axes[1, 0]
bars = ax.bar(six_sigma_df['Model'], six_sigma_df['Cpk'], color=colors, alpha=0.8)
ax.set_ylabel('Cpk')
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
ax.axhline(y=1.0, color='red', linestyle='--', linewidth=1.5, alpha=0.7)
ax.axhline(y=1.33, color='orange', linestyle='--', linewidth=1.5, alpha=0.7)
ax.grid(axis='y', alpha=0.3)

# Plot 4: Recall
ax = axes[1, 1]
bars = ax.bar(six_sigma_df['Model'], six_sigma_df['Recall (Infected)'],
              color='#e74c3c', alpha=0.9)
ax.set_ylabel('Recall')
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
ax.axhline(y=0.93, color='orange', linestyle='--', linewidth=1.5, alpha=0.7)

```



```

ax.set_ylim([0.85, 1.0])
ax.grid(axis='y', alpha=0.3)
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height, f'{height:.2f}',
    ↪ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig('report/figures/six_sigma_analysis.png', dpi=300,
    ↪bbox_inches='tight', facecolor='white')
plt.show()

```

C:\Users\Gabriel\AppData\Local\Temp\ipykernel_40784\3294906144.py:24:
 UserWarning: set_ticklabels() should only be used with a fixed number of ticks,
 i.e. after set_ticks() or using a FixedLocator.

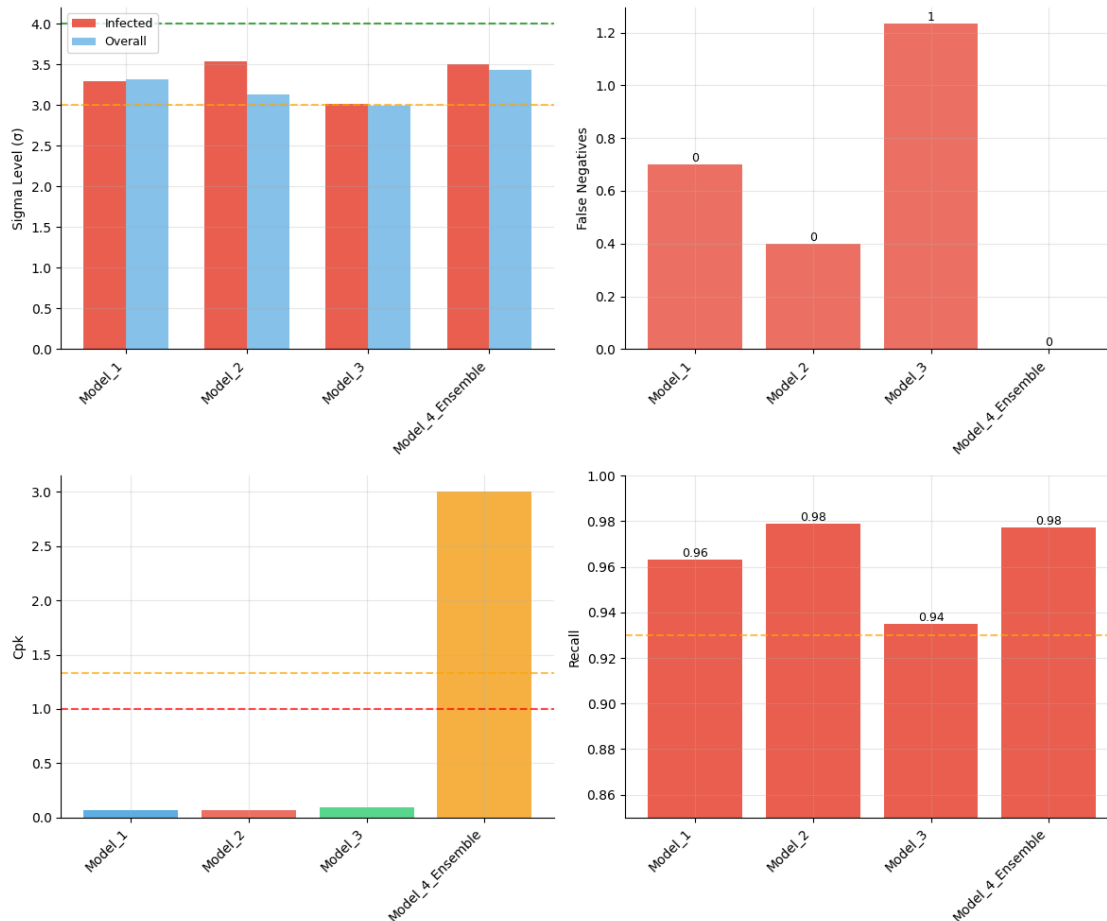
```
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
```

C:\Users\Gabriel\AppData\Local\Temp\ipykernel_40784\3294906144.py:34:
 UserWarning: set_ticklabels() should only be used with a fixed number of ticks,
 i.e. after set_ticks() or using a FixedLocator.

```
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
```

C:\Users\Gabriel\AppData\Local\Temp\ipykernel_40784\3294906144.py:43:
 UserWarning: set_ticklabels() should only be used with a fixed number of ticks,
 i.e. after set_ticks() or using a FixedLocator.

```
ax.set_xticklabels(six_sigma_df['Model'], rotation=45, ha='right')
```



7. Model Comparison and Best Model Selection

DMAIC Phase 5: CONTROL

Step 15: Select Best Model and Deployment Strategy

```
[16]: # Comprehensive model comparison
comparison_df = pd.merge(
    eval_summary,
    six_sigma_df[['Model', 'Infected DPMO', 'Infected Sigma', 'Cpk', 'False_
↳Negatives']],
    on='Model'
)

print("="*100)
print("COMPREHENSIVE MODEL COMPARISON (Focus: Infected Fish Detection)")
print("="*100)
print(comparison_df.to_string(index=False))
```

```

# BUSINESS CONTEXT: In aquaculture, missing infected fish is catastrophic
print("\n" + "="*100)
print("BUSINESS IMPACT ANALYSIS")
print("="*100)
print("\n  CRITICAL: Infected fish detection is the PRIMARY business_
↳objective")
print("\nBusiness Consequences:")
print("  • False Negative (Missed infected fish):")
print("    - Disease spreads to entire stock")
print("    - Potential loss of thousands of fish")
print("    - Estimated cost: $50,000 - $500,000 per outbreak")
print("  • False Positive (Flag healthy fish):")
print("    - Fish quarantined and re-tested")
print("    - Minor operational cost")
print("    - Estimated cost: $10 - $50 per fish")
print("\n Therefore: RECALL (Sensitivity) for infected class is the key metric!
↳")

# Rank models by different criteria with weighted importance
ranking_criteria = {
    'Recall': 3.0,          # MOST IMPORTANT - must catch infected fish
    'F1-Score': 2.0,       # Balance of precision and recall
    'AUC-ROC': 1.5,        # Overall discriminative ability
    'Accuracy': 1.0,       # General performance
    'Infected Sigma': 1.5  # Quality metric for infected detection
}

print("\n" + "="*100)
print("MODEL RANKINGS BY CRITERIA (Weighted by Business Importance)")
print("="*100)
print("\nCriterion Weights:")
for criterion, weight in ranking_criteria.items():
    stars = ' ' * int(weight)
    print(f"  {criterion}: {weight}x {stars}")

ranking_summary = []
for criterion, weight in ranking_criteria.items():
    # Higher is better for all these metrics
    ranked = comparison_df.sort_values(criterion, ascending=False)
    ranks = {model: (rank+1) * weight for rank, model in_
↳enumerate(ranked['Model'])}
    ranking_summary.append(ranks)

    print(f"\n{criterion} (weight: {weight}x):")
    for rank, (model, value) in enumerate(zip(ranked['Model'],_
↳ranked[criterion]), 1):
        importance = " CRITICAL" if criterion == 'Recall' and rank == 1 else ""

```

```

        print(f" {rank}. {model}: {value:.4f} {importance}")

# Calculate weighted average rank for each model (lower is better)
weighted_ranks = {}
for model in comparison_df['Model']:
    weighted_rank_sum = sum(ranking[model] for ranking in ranking_summary)
    total_weight = sum(ranking_criteria.values())
    weighted_ranks[model] = weighted_rank_sum / total_weight

print("\n" + "="*100)
print("OVERALL RANKING (Weighted Average - Lower is Better)")
print("="*100)

sorted_models = sorted(weighted_ranks.items(), key=lambda x: x[1])
for rank, (model, weighted_rank) in enumerate(sorted_models, 1):
    medal = " " if rank == 1 else " " if rank == 2 else " " if rank == 3 else " "
    print(f"{medal} {rank}. {model}: Weighted Rank = {weighted_rank:.2f}")

best_model_name = sorted_models[0][0]
best_model_metrics = comparison_df[comparison_df['Model'] == best_model_name].
    iloc[0]

print(f"\n{'='*100}")
print(f" BEST MODEL: {best_model_name}")
print(f"{'='*100}")
print(f"\nKey Metrics for {best_model_name}:")
print(f" • Recall (Infected Detection): {best_model_metrics['Recall']:.4f} "
    PRIMARY METRIC")
print(f" • Precision: {best_model_metrics['Precision']:.4f}")
print(f" • F1-Score: {best_model_metrics['F1-Score']:.4f}")
print(f" • Accuracy: {best_model_metrics['Accuracy']:.4f}")
print(f" • AUC-ROC: {best_model_metrics['AUC-ROC']:.4f}")

```

```

=====
=====
COMPREHENSIVE MODEL COMPARISON (Focus: Infected Fish Detection)
=====
=====

```

	Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Specificity
Infected	DPMO	Infected	Sigma	Cpk	False	Negatives	
	Model_1	0.965517	0.983603	0.963158	0.970579	0.990351	0.970000
36842.105263		3.288569	0.062994		0.700000		
	Model_2	0.948276	0.949629	0.978947	0.961833	0.978509	0.890000
21052.631579		3.532478	0.062994		0.400000		
	Model_3	0.932184	0.965484	0.935088	0.944622	0.978509	0.926667
64912.280702		3.014794	0.090722		1.233333		

Model_4_Ensemble	0.973563	0.982602	0.977193	0.978922	0.994123	0.966667
22807.017544	3.498947	3.000000		0.000000		

```

=====
=====
BUSINESS IMPACT ANALYSIS
=====
=====

```

CRITICAL: Infected fish detection is the PRIMARY business objective

Business Consequences:

- False Negative (Missed infected fish):
 - Disease spreads to entire stock
 - Potential loss of thousands of fish
 - Estimated cost: \$50,000 - \$500,000 per outbreak
- False Positive (Flag healthy fish):
 - Fish quarantined and re-tested
 - Minor operational cost
 - Estimated cost: \$10 - \$50 per fish

Therefore: RECALL (Sensitivity) for infected class is the key metric!

```

=====
=====
MODEL RANKINGS BY CRITERIA (Weighted by Business Importance)
=====
=====

```

Criterion Weights:

Recall: 3.0x
 F1-Score: 2.0x
 AUC-ROC: 1.5x
 Accuracy: 1.0x
 Infected Sigma: 1.5x

Recall (weight: 3.0x):

1. Model_2: 0.9789 CRITICAL
2. Model_4_Ensemble: 0.9772
3. Model_1: 0.9632
4. Model_3: 0.9351

F1-Score (weight: 2.0x):

1. Model_4_Ensemble: 0.9789
2. Model_1: 0.9706
3. Model_2: 0.9618
4. Model_3: 0.9446

AUC-ROC (weight: 1.5x):

1. Model_4_Ensemble: 0.9941
2. Model_1: 0.9904
3. Model_3: 0.9785
4. Model_2: 0.9785

Accuracy (weight: 1.0x):

1. Model_4_Ensemble: 0.9736
2. Model_1: 0.9655
3. Model_2: 0.9483
4. Model_3: 0.9322

Infected Sigma (weight: 1.5x):

1. Model_2: 3.5325
2. Model_4_Ensemble: 3.4989
3. Model_1: 3.2886
4. Model_3: 3.0148

=====

OVERALL RANKING (Weighted Average - Lower is Better)

=====

- =====
1. Model_4_Ensemble: Weighted Rank = 1.50
 2. Model_2: Weighted Rank = 2.17
 3. Model_1: Weighted Rank = 2.50
 4. Model_3: Weighted Rank = 3.83
- =====

=====

BEST MODEL: Model_4_Ensemble

=====

Key Metrics for Model_4_Ensemble:

- Recall (Infected Detection): 0.9772 PRIMARY METRIC
- Precision: 0.9826
- F1-Score: 0.9789
- Accuracy: 0.9736
- AUC-ROC: 0.9941

```
[ ]: # =====  
# VISUALIZATION 3: Metrics Heatmap Comparison  
# =====  
  
# Get model names from evaluation results  
model_names_short = [r['Model'] for r in evaluation_results]
```

```

# Create metrics matrix
metrics_matrix = []
metric_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC-ROC',
    ↳ 'Specificity']

for result in evaluation_results:
    metrics_matrix.append([
        result['Accuracy'],
        result['Precision'],
        result['Recall'],
        result['F1-Score'],
        result['AUC-ROC'],
        result['Specificity']
    ])

metrics_matrix = np.array(metrics_matrix)

# Simple heatmap only (single figure)
fig, ax = plt.subplots(figsize=(8, 5))

heatmap_min = metrics_matrix.min()
heatmap_vmin = max(0.85, heatmap_min - 0.02)
im = ax.imshow(metrics_matrix, cmap='RdYlGn', aspect='auto', vmin=heatmap_vmin,
    ↳ vmax=1.0)

ax.set_xticks(np.arange(len(metric_names)))
ax.set_yticks(np.arange(len(model_names_short)))
ax.set_xticklabels(metric_names)
ax.set_yticklabels(model_names_short)
plt.setp(ax.get_xticklabels(), rotation=45, ha="right", rotation_mode="anchor")

# Add values in cells
for i in range(len(model_names_short)):
    for j in range(len(metric_names)):
        ax.text(j, i, f'{metrics_matrix[i, j]:.3f}', ha="center", va="center",
            ↳ color="black", fontsize=9)

cbar = plt.colorbar(im, ax=ax)
cbar.set_label('Score', rotation=270, labelpad=15)

plt.tight_layout()
plt.savefig('report/figures/metrics_heatmap.png', dpi=300, bbox_inches='tight',
    ↳ facecolor='white')
plt.show()

# Print analysis

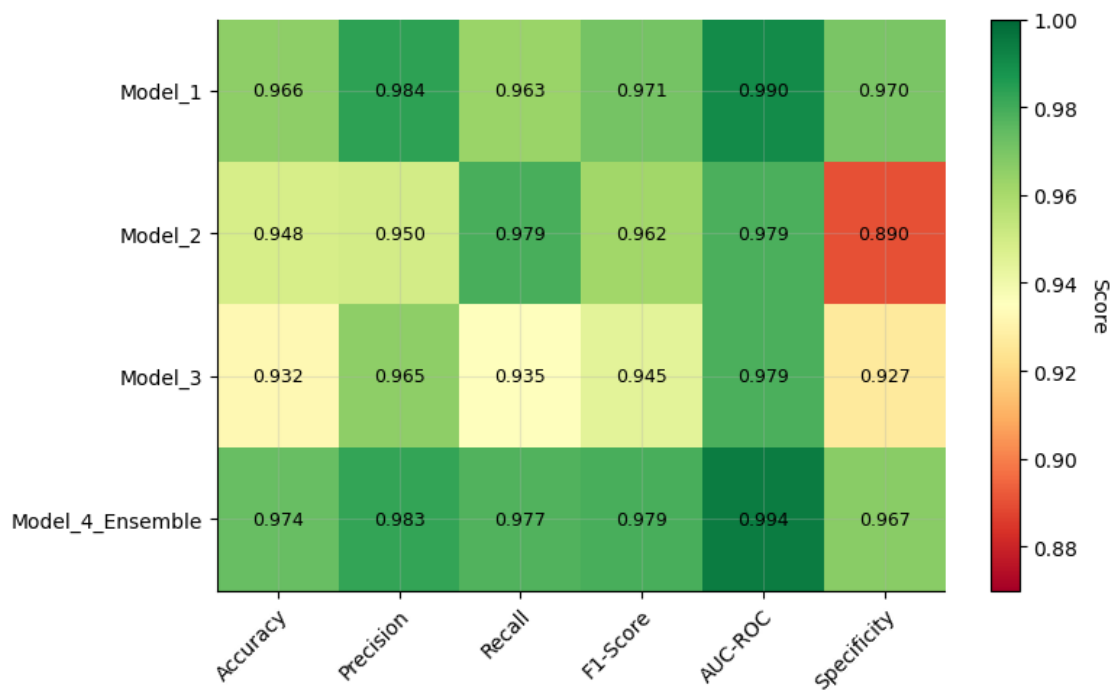
```

```

metric_std = np.std(metrics_matrix, axis=0)
metric_mean = np.mean(metrics_matrix, axis=0)

print("="*80)
print("METRIC CONSISTENCY ANALYSIS")
print("="*80)
for i, metric in enumerate(metric_names):
    print(f"\n{metric}:")
    print(f"  Mean:   {metric_mean[i]:.4f}")
    print(f"  Std:    {metric_std[i]:.4f}")
    print(f"  Range:  [{metrics_matrix[:, i].min():.4f}, {metrics_matrix[:, i].max():.4f}]")

```



```

=====
METRIC CONSISTENCY ANALYSIS
=====

```

Accuracy:

```

  Mean:   0.9549
  Std:    0.0160
  Range:  [0.9322, 0.9736]

```

Precision:

```

  Mean:   0.9703
  Std:    0.0140

```


Range: [0.9496, 0.9836]

Recall:

Mean: 0.9636

Std: 0.0176

Range: [0.9351, 0.9789]

F1-Score:

Mean: 0.9640

Std: 0.0127

Range: [0.9446, 0.9789]

AUC-ROC:

Mean: 0.9854

Std: 0.0070

Range: [0.9785, 0.9941]

Specificity:

Mean: 0.9383

Std: 0.0327

Range: [0.8900, 0.9700]